

# Deep Learning Mastery in 5 Weeks

Complete Notebook Collection

12 Jupyter Notebooks + Cheat Sheet + Quick Reference Card

Neural Networks | Deep Learning | Generative AI | GNNs | Transformers

# Table of Contents

---

## Week 1 — Build a Brain from Scratch (Pure Python)

- W1\_01 Single Neuron — activations, forward pass, AND/XOR gates
- W1\_02 Backpropagation — chain rule, gradient checking, solving XOR
- W1\_03 Multi-Layer Network — UAT, depth experiments, real datasets

## Week 2 — Go Deep with PyTorch

- W2\_01 PyTorch First Network — rebuilding Week 1, optimizer comparison
- W2\_02 Going Deeper — He/Xavier init, BatchNorm, loss landscapes

## Week 3 — Generate and Ship

- W3\_01 Autoencoder & VAE from Scratch — pure NumPy generative model
- W3\_02 PyTorch Generative AI — VAE on images, latent space, generation
- W3\_03 Deploy Your Model — Gradio web app, Hugging Face Spaces

## Week 4 — Graph Neural Networks

- W4\_01 GNN from Scratch — adjacency matrices, message passing, GCN
- W4\_02 PyTorch GNN — GraphSAGE, over-smoothing, real graph datasets

## Week 5 — Transformers

- W5\_01 Attention & Transformers from Scratch — Q/K/V, multi-head, positional encoding
- W5\_02 PyTorch Transformer — encoder classifier, causal masking, BERT vs GPT

## Bonus

- Cheat Sheet — every key pattern from the course in one document
- Quick Reference Card — printable single-page reference

# Week 1, Notebook 1: The Single Neuron

---

## Pure Python + NumPy — No Frameworks

**What you'll build:** A single neuron from scratch, trained with gradient descent.

**Curriculum points:**

- ② ReLU changed everything (sigmoid vs ReLU comparison)
- ④ Neural networks don't "understand" — they minimize loss
- ⑤ Backpropagation is just the chain rule

**Time estimate:** 30–45 minutes

---

### Key Insight

A neuron is just:  $\text{output} = \text{activation}(\text{weights} \cdot \text{inputs} + \text{bias})$  That's it. Everything else is optimization.

## Part 1: Build a Single Neuron

In [1]

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 np.random.seed(42)
4
5 # =====
6 # A single neuron: the atom of deep learning
7 # =====
8 # A neuron computes:  $z = w \cdot x + b$ , then  $a = \text{activation}(z)$ 
9 # Let's build one from absolute zero.
10
11 class Neuron:
12     """A single artificial neuron."""
13
14     def __init__(self, n_inputs, activation='sigmoid'):
15         # Initialize weights randomly (small values)
16         self.w = np.random.randn(n_inputs) * 0.1
17         self.b = 0.0
18         self.activation = activation
19
20         # Storage for backprop
21         self.z = None # pre-activation
22         self.a = None # post-activation
23         self.x = None # input (cached)
24
25     def _activate(self, z):
26         """Apply activation function."""
27         if self.activation == 'sigmoid':
28             return 1.0 / (1.0 + np.exp(-np.clip(z, -500, 500)))
29         elif self.activation == 'relu':
30             return np.maximum(0, z)
31         elif self.activation == 'linear':
32             return z
33
34     def _activate_derivative(self, z):
35         """Derivative of activation - needed for backprop."""
36         if self.activation == 'sigmoid':
37             s = self._activate(z)
38             return s * (1 - s)
39         elif self.activation == 'relu':
40             return (z > 0).astype(float)
41         elif self.activation == 'linear':
42             return np.ones_like(z)
43
44     def forward(self, x):
45         """Forward pass: compute output."""
46         self.x = x
47         self.z = np.dot(x, self.w) + self.b # linear combination
48         self.a = self._activate(self.z) # apply activation
49         return self.a
50
51     def __repr__(self):
52         return f"Neuron(w={self.w}, b={self.b:.4f}, act={self.activation})"
53
54 # Create a neuron with 2 inputs
55 neuron = Neuron(n_inputs=2, activation='sigmoid')
56 print("Our neuron:", neuron)
57
58 # Pass some data through it
59
60 x_sample = np.array([1.0, 0.5])
61 output = neuron.forward(x_sample)
62 print(f"Input: {x_sample} → Output: {output:.4f}")

```

## ■ What just happened?

The neuron: 1. Multiplied each input by its weight 2. Added the bias 3. Squeezed the result through sigmoid (maps everything to 0–1)

**Key question:** *Can this single neuron learn anything useful?*

Let's find out by training it on logic gates.

## Part 2: Train the Neuron — AND Gate

In [2]

```

1 # =====
2 # Training data: AND gate
3 # =====
4 # AND: output is 1 only when BOTH inputs are 1
5 X_and = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
6 y_and = np.array([0, 0, 0, 1])
7
8 # Loss function: Mean Squared Error
9 def mse_loss(predicted, actual):
10     return np.mean((predicted - actual) ** 2)
11
12 # =====
13 # Training loop – gradient descent by hand
14 # =====
15 def train_neuron(neuron, X, y, learning_rate=0.5, epochs=1000, verbose=True):
16     """Train a single neuron using gradient descent.
17
18     This is backprop for ONE neuron:
19     - Loss = (1/N)  $\sum (a - y)^2$ 
20     - dL/da = 2(a - y) / N
21     - da/dz = activation'(z)      ← chain rule!
22     - dz/dw = x                    ← chain rule again!
23     - dz/db = 1
24
25     So: dL/dw = dL/da · da/dz · dz/dw = dL/da · activation'(z) · x
26     """
27     losses = []
28
29     for epoch in range(epochs):
30         # Forward pass (all samples at once)
31         predictions = np.array([neuron.forward(x) for x in X])
32         loss = mse_loss(predictions, y)
33         losses.append(loss)
34
35         # Backward pass – compute gradients
36         dw = np.zeros_like(neuron.w)
37         db = 0.0
38
39         for i in range(len(X)):
40             neuron.forward(X[i]) # recompute (sets cached values)
41
42             # Chain rule in action:
43             dL_da = 2 * (neuron.a - y[i]) / len(X) # loss gradient
44             da_dz = neuron._activate_derivative(neuron.z) # activation gradient
45             dz_dw = neuron.x # input
46
47             # Combine via chain rule: dL/dw = dL/da · da/dz · dz/dw
48             dw += dL_da * da_dz * dz_dw
49             db += dL_da * da_dz * 1.0
50
51         # Update weights (gradient descent step)
52         neuron.w -= learning_rate * dw
53         neuron.b -= learning_rate * db
54
55         if verbose and epoch % 200 == 0:
56             print(f"Epoch {epoch:4d} | Loss: {loss:.6f} | w: {neuron.w} | b: {neuron.b:.4f}")
57
58     return losses

```

```

59
60 # Train!
61 neuron_and = Neuron(2, activation='sigmoid')
62 losses = train_neuron(neuron_and, X_and, y_and, learning_rate=2.0, epochs=2000)
63
64 # Test
65 print("\n--- AND Gate Results ---")
66 for x, y_true in zip(X_and, y_and):
67     pred = neuron_and.forward(x)
68     print(f" {x} → {pred:.4f} (target: {y_true})")

```

In [3]

```

1 # Visualize training
2 fig, axes = plt.subplots(1, 2, figsize=(12, 4))
3
4 # Loss curve
5 axes[0].plot(losses)
6 axes[0].set_xlabel('Epoch')
7 axes[0].set_ylabel('MSE Loss')
8 axes[0].set_title('Training Loss – AND Gate')
9 axes[0].set_yscale('log')
10 axes[0].grid(True, alpha=0.3)
11
12 # Decision boundary
13 xx, yy = np.meshgrid(np.linspace(-0.5, 1.5, 200), np.linspace(-0.5, 1.5, 200))
14 grid = np.c_[xx.ravel(), yy.ravel()]
15 zz = np.array([neuron_and.forward(p) for p in grid]).reshape(xx.shape)
16
17 axes[1].contourf(xx, yy, zz, levels=20, cmap='RdYlBu_r', alpha=0.8)
18 axes[1].contour(xx, yy, zz, levels=[0.5], colors='black', linewidths=2)
19 for x, y_true in zip(X_and, y_and):
20     color = 'red' if y_true == 1 else 'blue'
21     axes[1].scatter(x[0], x[1], c=color, s=200, edgecolors='black', zorder=5)
22 axes[1].set_title('Decision Boundary – AND Gate')
23 axes[1].set_xlabel('x■')
24 axes[1].set_ylabel('x■')
25
26 plt.tight_layout()
27 plt.savefig('w1_01_and_gate.png', dpi=100, bbox_inches='tight')
28 plt.show()
29 print("✓ Saved figure")

```

## Part 3: Sigmoid vs. ReLU — Why ReLU Changed Everything

**Curriculum Point ②:** Before ReLU, sigmoid and tanh dominated. The problem: *vanishing gradients*.

Let's SEE this problem.

In [4]

```

1 # =====
2 # SIGMOID vs RELU: visualize the gradient problem
3 # =====
4 z = np.linspace(-6, 6, 300)
5
6 fig, axes = plt.subplots(2, 2, figsize=(12, 8))
7
8 # Sigmoid and its derivative
9 sigmoid = 1 / (1 + np.exp(-z))
10 sigmoid_grad = sigmoid * (1 - sigmoid)
11 axes[0, 0].plot(z, sigmoid, 'b-', linewidth=2)
12 axes[0, 0].set_title('Sigmoid Activation')
13 axes[0, 0].grid(True, alpha=0.3)
14 axes[0, 0].axhline(y=0, color='k', linewidth=0.5)
15
16 axes[0, 1].plot(z, sigmoid_grad, 'b-', linewidth=2)
17 axes[0, 1].set_title('Sigmoid Gradient')
18 axes[0, 1].axhline(y=0.25, color='r', linestyle='--', alpha=0.5, label='max = 0.25')
19 axes[0, 1].legend()
20 axes[0, 1].grid(True, alpha=0.3)
21
22 # ReLU and its derivative
23 relu = np.maximum(0, z)
24 relu_grad = (z > 0).astype(float)
25 axes[1, 0].plot(z, relu, 'r-', linewidth=2)
26 axes[1, 0].set_title('ReLU Activation')
27 axes[1, 0].grid(True, alpha=0.3)
28 axes[1, 0].axhline(y=0, color='k', linewidth=0.5)
29
30 axes[1, 1].plot(z, relu_grad, 'r-', linewidth=2)
31 axes[1, 1].set_title('ReLU Gradient')
32 axes[1, 1].set_ylim(-0.1, 1.3)
33 axes[1, 1].grid(True, alpha=0.3)
34
35 plt.suptitle('WHY ReLU CHANGED EVERYTHING', fontsize=14, fontweight='bold')
36 plt.tight_layout()
37 plt.savefig('w1_01_relu_vs_sigmoid.png', dpi=100, bbox_inches='tight')
38 plt.show()
39
40 print("KEY INSIGHT:")
41 print(f" Sigmoid max gradient: {sigmoid_grad.max():.2f} – multiplied across layers, this VANISHES")
42 print(f" ReLU gradient (when active): 1.00 – gradients flow unchanged!")
43 print(f" This is why deep networks couldn't train before ReLU.")

```

## Part 4: The XOR Challenge

Can a single neuron learn XOR? Try it and observe the failure.

XOR: output is 1 when inputs DIFFER.

In [5]

```

1 # =====
2 # XOR gate – THE classic single-neuron failure case
3 # =====
4 X_xor = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
5 y_xor = np.array([0, 1, 1, 0])
6
7 neuron_xor = Neuron(2, activation='sigmoid')
8 losses_xor = train_neuron(neuron_xor, X_xor, y_xor, learning_rate=2.0, epochs=5000, verbose=False)
9
10 print("--- XOR Gate Results (Single Neuron) ---")
11 for x, y_true in zip(X_xor, y_xor):
12     pred = neuron_xor.forward(x)
13     print(f" {x} → {pred:.4f} (target: {y_true})")
14
15 print(f"\nFinal loss: {losses_xor[-1]:.6f}")
16 print("\n■■■ The neuron CANNOT learn XOR!")
17 print("    A single neuron draws a LINEAR decision boundary.")
18 print("    XOR is not linearly separable.")
19 print("    → This is why we need MULTIPLE LAYERS (next notebook).")
20
21 # Visualize the failure
22 fig, axes = plt.subplots(1, 2, figsize=(12, 4))
23
24 axes[0].plot(losses_xor)
25 axes[0].set_title('XOR Training Loss (stuck!)')
26 axes[0].set_xlabel('Epoch')
27 axes[0].set_ylabel('Loss')
28 axes[0].grid(True, alpha=0.3)
29
30 xx, yy = np.meshgrid(np.linspace(-0.5, 1.5, 200), np.linspace(-0.5, 1.5, 200))
31 grid = np.c_[xx.ravel(), yy.ravel()]
32 zz = np.array([neuron_xor.forward(p) for p in grid]).reshape(xx.shape)
33 axes[1].contourf(xx, yy, zz, levels=20, cmap='RdYlBu_r', alpha=0.8)
34 for x, y_true in zip(X_xor, y_xor):
35     color = 'red' if y_true == 1 else 'blue'
36     axes[1].scatter(x[0], x[1], c=color, s=200, edgecolors='black', zorder=5)
37 axes[1].set_title('XOR: No Linear Boundary Exists')
38 axes[1].set_xlabel('x■■')
39 axes[1].set_ylabel('x■■')
40
41 plt.tight_layout()
42 plt.savefig('w1_01_xor_failure.png', dpi=100, bbox_inches='tight')
43 plt.show()

```

## ■ Self-Check: "You know you've got it when..."

- [ ] You can explain what  $z = w \cdot x + b$  computes geometrically (a hyperplane)
- [ ] You can explain in ONE sentence why ReLU fixes vanishing gradients
  - *Answer: ReLU's gradient is either 0 or 1 — it never shrinks, so gradients don't vanish when multiplied across layers.*
- [ ] You understand why XOR fails: a single neuron = a single linear boundary
- [ ] You wrote the gradient descent update rule without looking

## ■ Challenge (5 min)

Modify the code above to train an OR gate and a NAND gate. Compare the learned weights.

➡■ **Next:** `w1_02_Pure_Backprop_From_Scratch.ipynb` — **Multi-neuron backpropagation**

# Week 1, Notebook 2: Backpropagation from Scratch

## Building the Engine of Deep Learning

**What you'll build:** A complete backpropagation engine for multi-layer networks.

**Curriculum points:**

- ⑤ Backpropagation is just the chain rule (dynamic programming for gradients)
- ② ReLU for better gradient flow
- ④ Neural networks minimize loss, not "understand"

**Time estimate:** 45–60 minutes

### The Big Idea

Backprop isn't magic. It's the chain rule applied systematically from output back to input. Think of it as: "How much did each weight contribute to the error?"

```
1 Forward: input → layer1 → layer2 → output → loss
2 Backward: input ← layer1 ← layer2 ← output ← loss
3           gradients flow backward
```

## Part 1: The Chain Rule — Visualized

In [1]

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 np.random.seed(42)
4
5 # =====
6 # Chain rule refresher with a concrete example
7 # =====
8 # Suppose:  $L = (\text{relu}(w \cdot x + b) - y)^2$ 
9 #
10 # Forward:  $z = w \cdot x + b \rightarrow a = \text{relu}(z) \rightarrow L = (a - y)^2$ 
11 #
12 # Backward (chain rule):
13 #  $dL/da = 2(a - y)$ 
14 #  $da/dz = 1$  if  $z > 0$ , else  $0$  (ReLU derivative)
15 #  $dz/dw = x$ 
16 #  $dz/db = 1$ 
17 #
18 #  $dL/dw = dL/da \cdot da/dz \cdot dz/dw = 2(a-y) \cdot \text{relu}'(z) \cdot x$ 
19 #  $dL/db = dL/da \cdot da/dz \cdot dz/db = 2(a-y) \cdot \text{relu}'(z) \cdot 1$ 
20
21 # Let's verify with numerical gradients
22 def numerical_gradient(f, param, idx, eps=1e-5):
23     """Compute gradient numerically (for verification)."""
24     original = param[idx].copy() if hasattr(param[idx], 'copy') else param[idx]
25
26     param[idx] = original + eps
27     loss_plus = f()
28
29     param[idx] = original - eps
30     loss_minus = f()
31
32     param[idx] = original # restore
33     return (loss_plus - loss_minus) / (2 * eps)
34
35 # Simple example
36 w, x, b, y = 0.5, 2.0, 0.1, 1.0
37
38 # Forward
39 z = w * x + b
40 a = max(0, z) # ReLU
41 L = (a - y) ** 2
42
43 # Analytical gradients (chain rule)
44 dL_da = 2 * (a - y)
45 da_dz = 1.0 if z > 0 else 0.0
46 dz_dw = x
47 dz_db = 1.0
48
49 dL_dw_analytical = dL_da * da_dz * dz_dw
50 dL_db_analytical = dL_da * da_dz * dz_db
51
52 # Numerical verification
53 params = [w]
54 def loss_fn_w():
55     z = params[0] * x + b
56     a = max(0, z)
57     return (a - y) ** 2

```

```

59 dL_dw_numerical = numerical_gradient(loss_fn_w, params, 0)
60
61 print("Chain Rule Verification:")
62 print(f" Forward: z={z:.4f}, a={a:.4f}, L={L:.4f}")
63 print(f" dL/dw (analytical): {dL_dw_analytical:.6f}")
64 print(f" dL/dw (numerical): {dL_dw_numerical:.6f}")
65 print(f" Match: {np.isclose(dL_dw_analytical, dL_dw_numerical, atol=1e-4)}")
66 print(f"\n dL/db (analytical): {dL_db_analytical:.6f}")
67 print("\n✓ Backprop IS the chain rule. That's all there is to it.")

```

## Part 2: Full Multi-Layer Network with Backprop

Now we build a **complete neural network class** with proper forward AND backward passes.

This is the code that will let us solve XOR (which a single neuron couldn't).

In [2]

```

1 # =====
2 # A complete neural network from scratch
3 # =====
4
5 class Layer:
6     """A single fully-connected layer."""
7
8     def __init__(self, n_in, n_out, activation='relu'):
9         # He initialization for ReLU, Xavier for sigmoid
10         if activation == 'relu':
11             scale = np.sqrt(2.0 / n_in) # He init
12         else:
13             scale = np.sqrt(1.0 / n_in) # Xavier init
14
15         self.W = np.random.randn(n_in, n_out) * scale
16         self.b = np.zeros(n_out)
17         self.activation = activation
18
19         # Cache for backprop
20         self.x = None
21         self.z = None
22
23         # Gradients
24         self.dW = None
25         self.db = None
26
27     def forward(self, x):
28         """Forward pass."""
29         self.x = x # cache input
30         self.z = x @ self.W + self.b # linear transform
31
32         # Apply activation
33         if self.activation == 'relu':
34             return np.maximum(0, self.z)
35         elif self.activation == 'sigmoid':
36             return 1.0 / (1.0 + np.exp(-np.clip(self.z, -500, 500)))
37         elif self.activation == 'linear':
38             return self.z
39
40     def backward(self, da):
41         """Backward pass – compute gradients.
42
43         da: gradient of loss w.r.t. this layer's output (activation)
44         Returns: gradient of loss w.r.t. this layer's INPUT (for previous layer)
45         """
46         # Step 1: gradient through activation
47         if self.activation == 'relu':
48             dz = da * (self.z > 0).astype(float)
49         elif self.activation == 'sigmoid':
50             s = 1.0 / (1.0 + np.exp(-np.clip(self.z, -500, 500)))
51             dz = da * s * (1 - s)
52         elif self.activation == 'linear':
53             dz = da
54
55         # Step 2: gradients for parameters
56         m = self.x.shape[0] # batch size
57         self.dW = (self.x.T @ dz) / m # dL/dW
58         self.db = np.mean(dz, axis=0) # dL/db

```

```

59
60     # Step 3: gradient for input (to pass to previous layer)
61     dx = dz @ self.W.T
62     return dx
63
64
65 class NeuralNetwork:
66     """Multi-layer neural network with backpropagation."""
67
68     def __init__(self, layer_sizes, activations=None):
69         """
70         layer_sizes: e.g., [2, 4, 4, 1] for 2 inputs, two hidden layers of 4, 1 output
71         activations: e.g., ['relu', 'relu', 'sigmoid']
72         """
73         if activations is None:
74             activations = ['relu'] * (len(layer_sizes) - 2) + ['sigmoid']
75
76         self.layers = []
77         for i in range(len(layer_sizes) - 1):
78             self.layers.append(Layer(layer_sizes[i], layer_sizes[i+1], activations[i]))
79
80     def forward(self, X):
81         """Forward pass through all layers."""
82         out = X
83         for layer in self.layers:
84             out = layer.forward(out)
85         return out
86
87     def backward(self, y_pred, y_true):
88         """Backward pass – backpropagate gradients through all layers."""
89         # Start with loss gradient (MSE:  $dL/da = 2(a - y) / N$ )
90         m = y_true.shape[0]
91         da = 2 * (y_pred - y_true) / m
92
93         # Propagate backward through layers (REVERSE order!)
94         for layer in reversed(self.layers):
95             da = layer.backward(da)
96
97     def update(self, lr):
98         """Update all weights using computed gradients."""
99         for layer in self.layers:
100             layer.W -= lr * layer.dW
101             layer.b -= lr * layer.db
102
103     def train(self, X, y, lr=0.01, epochs=1000, verbose=True):
104         """Full training loop."""
105         losses = []
106         for epoch in range(epochs):
107             # Forward
108             y_pred = self.forward(X)
109
110             # Loss
111             loss = np.mean((y_pred - y.reshape(-1, 1)) ** 2)
112             losses.append(loss)
113
114             # Backward
115             self.backward(y_pred, y.reshape(-1, 1))

```

```
117         # Update
118         self.update(lr)
119
120         if verbose and epoch % (epochs // 5) == 0:
121             print(f" Epoch {epoch:5d} | Loss: {loss:.6f}")
122
123         return losses
124
125 # Quick test
126 net = NeuralNetwork([2, 4, 1], activations=['relu', 'sigmoid'])
127 print(f"Network architecture: {[l.W.shape for l in net.layers]}")
128 print("Ready for training!")
```

## Part 3: Solving XOR — The Multi-Layer Breakthrough

Remember: a single neuron FAILED at XOR because XOR isn't linearly separable. With two layers, we can solve it. Let's prove this.

In [3]

```

1 # =====
2 # XOR with a multi-layer network
3 # =====
4 X_xor = np.array([[0, 0], [0, 1], [1, 0], [1, 1]], dtype=float)
5 y_xor = np.array([0, 1, 1, 0], dtype=float)
6
7 # Architecture: 2 inputs → 8 hidden (ReLU) → 1 output (sigmoid)
8 net_xor = NeuralNetwork([2, 8, 1], activations=['relu', 'sigmoid'])
9
10 print("Training on XOR...")
11 losses = net_xor.train(X_xor, y_xor, lr=0.5, epochs=3000)
12
13 print("\n--- XOR Results (Multi-Layer Network) ---")
14 for x, y_true in zip(X_xor, y_xor):
15     pred = net_xor.forward(x.reshape(1, -1))[0, 0]
16     correct = "✓" if (pred > 0.5) == y_true else "✗"
17     print(f" {x} → {pred:.4f} (target: {y_true}) {correct}")
18
19 # Visualize
20 fig, axes = plt.subplots(1, 2, figsize=(12, 5))
21
22 axes[0].plot(losses)
23 axes[0].set_title('XOR Training Loss – SOLVED!')
24 axes[0].set_xlabel('Epoch')
25 axes[0].set_ylabel('MSE')
26 axes[0].set_yscale('log')
27 axes[0].grid(True, alpha=0.3)
28
29 xx, yy = np.meshgrid(np.linspace(-0.5, 1.5, 200), np.linspace(-0.5, 1.5, 200))
30 grid = np.c_[xx.ravel(), yy.ravel()]
31 zz = np.array([net_xor.forward(p.reshape(1, -1))[0, 0] for p in grid]).reshape(xx.shape)
32
33 axes[1].contourf(xx, yy, zz, levels=20, cmap='RdYlBu_r', alpha=0.8)
34 axes[1].contour(xx, yy, zz, levels=[0.5], colors='black', linewidths=2)
35 for x, y_true in zip(X_xor, y_xor):
36     color = 'red' if y_true == 1 else 'blue'
37     axes[1].scatter(x[0], x[1], c=color, s=200, edgecolors='black', zorder=5)
38 axes[1].set_title('XOR Decision Boundary – Non-Linear!')
39 axes[1].set_xlabel('x■')
40 axes[1].set_ylabel('x■')
41
42 plt.tight_layout()
43 plt.savefig('w1_02_xor_solved.png', dpi=100, bbox_inches='tight')
44 plt.show()
45 print("\n✓ Multiple layers create NON-LINEAR decision boundaries!")

```

## Part 4: Gradient Checking — Trust but Verify

Never trust your backprop implementation without checking it against numerical gradients.

In [4]

```

1 # =====
2 # Gradient checking: compare analytical vs numerical gradients
3 # =====
4 def gradient_check(net, X, y, eps=1e-5):
5     """Compare backprop gradients with numerical gradients."""
6     # First, do a normal forward + backward pass
7     y_pred = net.forward(X)
8     net.backward(y_pred, y.reshape(-1, 1))
9
10    print("Gradient Check (analytical vs numerical):")
11    print("-" * 55)
12
13    all_close = True
14    for layer_idx, layer in enumerate(net.layers):
15        # Check a few weights in each layer
16        for i in range(min(2, layer.W.shape[0])):
17            for j in range(min(2, layer.W.shape[1])):
18                # Numerical gradient
19                original = layer.W[i, j]
20
21                layer.W[i, j] = original + eps
22                loss_plus = np.mean((net.forward(X) - y.reshape(-1, 1)) ** 2)
23
24                layer.W[i, j] = original - eps
25                loss_minus = np.mean((net.forward(X) - y.reshape(-1, 1)) ** 2)
26
27                layer.W[i, j] = original # restore
28
29                numerical = (loss_plus - loss_minus) / (2 * eps)
30                analytical = layer.dw[i, j]
31
32                # Relative error
33                denom = max(abs(numerical) + abs(analytical), 1e-8)
34                rel_error = abs(numerical - analytical) / denom
35
36                ok = "✓" if rel_error < 1e-4 else "✗ MISMATCH"
37                if rel_error >= 1e-4:
38                    all_close = False
39
40                print(f" Layer {layer_idx} W[{i},{j}]: "
41                      f"analytical={analytical:+.6f} "
42                      f"numerical={numerical:+.6f} "
43                      f"rel_err={rel_error:.2e} {ok}")
44
45    print("-" * 55)
46    print(f"Overall: {'✓ All gradients match!' if all_close else '✗ Some gradients mismatch'}")
47    return all_close
48
49 # Test on our XOR network
50 net_check = NeuralNetwork([2, 4, 1], activations=['relu', 'sigmoid'])
51 gradient_check(net_check, X_xor, y_xor)

```

## ■ Self-Check

- [ ] You can draw the computation graph for forward AND backward pass
- [ ] You can explain backprop as "the chain rule applied backward through the network"
- [ ] Your gradient check passes (analytical  $\approx$  numerical)
- [ ] You solved XOR with a multi-layer network
- [ ] You understand WHY depth helps: each layer adds a non-linear transformation

➡■ **Next:** `w1_03_Pure_Multi_Layer_Network.ipynb` — **Real datasets, UAT, and depth experiments**

# Week 1, Notebook 3: Multi-Layer Networks on Real Data

## Universal Approximation & Depth Experiments

**What you'll build:** A configurable deep network trained on real data (moons/circles datasets).

**Curriculum points:**

- ① Universal Approximation Theorem: width vs depth tradeoff
- ⑦ Capacity  $\neq$  performance: more layers  $\neq$  better
- ② ReLU enables depth

**Time estimate:** 45–60 minutes

In [1]

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from collections import defaultdict
4 np.random.seed(42)
5
6 # =====
7 # Generate real-ish datasets (no sklearn needed)
8 # =====
9 def make_moons(n=300, noise=0.15):
10     """Generate two interleaving half-moons."""
11     t = np.linspace(0, np.pi, n // 2)
12     x1 = np.c_[np.cos(t), np.sin(t)] + np.random.randn(n // 2, 2) * noise
13     x2 = np.c_[np.cos(t) + 0.5, -np.sin(t) + 0.5] + np.random.randn(n // 2, 2) * noise
14     X = np.vstack([x1, x2])
15     y = np.hstack([np.zeros(n // 2), np.ones(n // 2)])
16     idx = np.random.permutation(n)
17     return X[idx], y[idx]
18
19 def make_circles(n=300, noise=0.08):
20     """Generate concentric circles."""
21     t = np.linspace(0, 2 * np.pi, n // 2)
22     x1 = np.c_[np.cos(t) * 0.5, np.sin(t) * 0.5] + np.random.randn(n // 2, 2) * noise
23     x2 = np.c_[np.cos(t) * 1.5, np.sin(t) * 1.5] + np.random.randn(n // 2, 2) * noise
24     X = np.vstack([x1, x2])
25     y = np.hstack([np.zeros(n // 2), np.ones(n // 2)])
26     idx = np.random.permutation(n)
27     return X[idx], y[idx]
28
29 # Generate data
30 X_moons, y_moons = make_moons(400, noise=0.15)
31 X_circles, y_circles = make_circles(400, noise=0.08)
32
33 # Normalize
34 for X in [X_moons, X_circles]:
35     X -= X.mean(axis=0)
36     X /= X.std(axis=0) + 1e-8
37
38 # Visualize
39 fig, axes = plt.subplots(1, 2, figsize=(12, 5))
40 for ax, X, y, title in [(axes[0], X_moons, y_moons, 'Moons'),
41                        (axes[1], X_circles, y_circles, 'Circles')]:
42     ax.scatter(X[y==0, 0], X[y==0, 1], c='steelblue', s=20, alpha=0.6, label='Class 0')
43     ax.scatter(X[y==1, 0], X[y==1, 1], c='coral', s=20, alpha=0.6, label='Class 1')
44     ax.set_title(f'{title} Dataset')
45     ax.legend()
46     ax.set_aspect('equal')
47     ax.grid(True, alpha=0.2)
48
49 plt.tight_layout()
50 plt.savefig('w1_03_datasets.png', dpi=100, bbox_inches='tight')
51 plt.show()

```

## Part 1: Reusable Network Class (improved from Notebook 2)

In [2]

```

1 # =====
2 # Improved Neural Network with train/test split and accuracy
3 # =====
4
5 class Layer:
6     def __init__(self, n_in, n_out, activation='relu'):
7         scale = np.sqrt(2.0 / n_in) if activation == 'relu' else np.sqrt(1.0 / n_in)
8         self.W = np.random.randn(n_in, n_out) * scale
9         self.b = np.zeros(n_out)
10        self.activation = activation
11        self.x = self.z = self.dW = self.db = None
12
13    def forward(self, x):
14        self.x = x
15        self.z = x @ self.W + self.b
16        if self.activation == 'relu':
17            return np.maximum(0, self.z)
18        elif self.activation == 'sigmoid':
19            return 1.0 / (1.0 + np.exp(-np.clip(self.z, -500, 500)))
20        return self.z
21
22    def backward(self, da):
23        if self.activation == 'relu':
24            dz = da * (self.z > 0).astype(float)
25        elif self.activation == 'sigmoid':
26            s = 1.0 / (1.0 + np.exp(-np.clip(self.z, -500, 500)))
27            dz = da * s * (1 - s)
28        else:
29            dz = da
30        m = self.x.shape[0]
31        self.dW = (self.x.T @ dz) / m
32        self.db = np.mean(dz, axis=0)
33        return dz @ self.W.T
34
35
36 class DeepNet:
37     def __init__(self, architecture):
38         """architecture: list of (n_neurons, activation) tuples, starting from input."""
39         self.layers = []
40         for i in range(len(architecture) - 1):
41             n_in = architecture[i][0]
42             n_out, act = architecture[i + 1]
43             self.layers.append(Layer(n_in, n_out, act))
44
45     def forward(self, X):
46         out = X
47         for layer in self.layers:
48             out = layer.forward(out)
49         return out
50
51     def backward(self, y_pred, y_true):
52         da = 2 * (y_pred - y_true) / y_true.shape[0]
53         for layer in reversed(self.layers):
54             da = layer.backward(da)
55
56     def update(self, lr):
57         for layer in self.layers:
58             layer.W -= lr * layer.dW

```

```

59         layer.b -= lr * layer.db
60
61     def count_params(self):
62         return sum(l.W.size + l.b.size for l in self.layers)
63
64     def fit(self, X_train, y_train, X_val, y_val, lr=0.01, epochs=2000):
65         history = {'train_loss': [], 'val_loss': [], 'train_acc': [], 'val_acc': []}
66         y_tr = y_train.reshape(-1, 1)
67         y_vl = y_val.reshape(-1, 1)
68
69         for epoch in range(epochs):
70             # Forward + backward on training data
71             pred_train = self.forward(X_train)
72             loss_train = np.mean((pred_train - y_tr) ** 2)
73             self.backward(pred_train, y_tr)
74             self.update(lr)
75
76             # Validation (forward only)
77             pred_val = self.forward(X_val)
78             loss_val = np.mean((pred_val - y_vl) ** 2)
79
80             # Accuracy
81             acc_train = np.mean(((pred_train > 0.5).astype(float) == y_tr).astype(float))
82             acc_val = np.mean(((pred_val > 0.5).astype(float) == y_vl).astype(float))
83
84             history['train_loss'].append(loss_train)
85             history['val_loss'].append(loss_val)
86             history['train_acc'].append(acc_train)
87             history['val_acc'].append(acc_val)
88
89         return history
90
91     # Train/test split helper
92     def split_data(X, y, ratio=0.8):
93         n = int(len(X) * ratio)
94         idx = np.random.permutation(len(X))
95         return X[idx[:n]], y[idx[:n]], X[idx[n:]], y[idx[n:]]
96
97     print("✓ DeepNet class ready for experiments")

```

## Part 2: Universal Approximation Theorem — Width Experiment

**Curriculum Point ①:** A single hidden layer can approximate any continuous function... but it may need an impractical number of neurons. Let's test this.

In [3]

```

1 # =====
2 # EXPERIMENT: How does WIDTH affect learning? (UAT in action)
3 # =====
4 X_tr, y_tr, X_val, y_val = split_data(X_moons, y_moons)
5
6 widths = [2, 4, 16, 64, 256]
7 results = {}
8
9 fig, axes = plt.subplots(2, len(widths), figsize=(20, 8))
10
11 for idx, w in enumerate(widths):
12     np.random.seed(42)
13     # Single hidden layer with varying width
14     arch = [(2, None), (w, 'relu'), (1, 'sigmoid')]
15     net = DeepNet(arch)
16
17     history = net.fit(X_tr, y_tr, X_val, y_val, lr=0.05, epochs=2000)
18     results[w] = history
19
20     # Decision boundary
21     xx, yy = np.meshgrid(np.linspace(-3, 3, 150), np.linspace(-3, 3, 150))
22     grid = np.c_[xx.ravel(), yy.ravel()]
23     zz = net.forward(grid).reshape(xx.shape)
24
25     axes[0, idx].contourf(xx, yy, zz, levels=20, cmap='RdYlBu_r', alpha=0.7)
26     axes[0, idx].scatter(X_val[y_val==0, 0], X_val[y_val==0, 1], c='steelblue', s=15, alpha=0.5)
27     axes[0, idx].scatter(X_val[y_val==1, 0], X_val[y_val==1, 1], c='coral', s=15, alpha=0.5)
28     axes[0, idx].set_title(f'Width={w} ({net.count_params()} params)')
29     axes[0, idx].set_xlim(-3, 3)
30     axes[0, idx].set_ylim(-3, 3)
31
32     axes[1, idx].plot(history['train_acc'], label='Train', alpha=0.8)
33     axes[1, idx].plot(history['val_acc'], label='Val', alpha=0.8)
34     axes[1, idx].set_ylim(0.4, 1.05)
35     axes[1, idx].set_xlabel('Epoch')
36     axes[1, idx].set_ylabel('Accuracy')
37     axes[1, idx].legend(fontsize=8)
38     axes[1, idx].grid(True, alpha=0.2)
39
40 plt.suptitle('UNIVERSAL APPROXIMATION: Width vs. Accuracy (1 Hidden Layer)',
41             fontsize=14, fontweight='bold')
42 plt.tight_layout()
43 plt.savefig('w1_03_uat_width.png', dpi=100, bbox_inches='tight')
44 plt.show()
45
46 print("\nFinal validation accuracies:")
47 for w, h in results.items():
48     print(f" Width {w:3d}: train={h['train_acc'][-1]:.3f}, val={h['val_acc'][-1]:.3f}")

```

## Part 3: Depth Experiment — Is Deeper Always Better?

**Curriculum Point ⑦:** Capacity  $\neq$  performance. More layers  $\neq$  better model.

In [4]

```

1 # =====
2 # EXPERIMENT: How does DEPTH affect learning?
3 # =====
4 X_tr, y_tr, X_val, y_val = split_data(X_circles, y_circles)
5
6 configs = [
7     ("1 layer (wide)", [(2, None), (32, 'relu'), (1, 'sigmoid')]),
8     ("2 layers", [(2, None), (16, 'relu'), (16, 'relu'), (1, 'sigmoid')]),
9     ("4 layers", [(2, None), (8, 'relu'), (8, 'relu'), (8, 'relu'), (8, 'relu'), (1, 'sigmoid')]),
10    ("8 layers", [(2, None)] + [(8, 'relu')] * 8 + [(1, 'sigmoid')]),
11]
12
13 fig, axes = plt.subplots(2, len(configs), figsize=(20, 8))
14
15 for idx, (name, arch) in enumerate(configs):
16     np.random.seed(42)
17     net = DeepNet(arch)
18     history = net.fit(X_tr, y_tr, X_val, y_val, lr=0.02, epochs=3000)
19
20     # Decision boundary
21     xx, yy = np.meshgrid(np.linspace(-3, 3, 150), np.linspace(-3, 3, 150))
22     grid = np.c_[xx.ravel(), yy.ravel()]
23     zz = net.forward(grid).reshape(xx.shape)
24
25     axes[0, idx].contourf(xx, yy, zz, levels=20, cmap='RdYlBu_r', alpha=0.7)
26     axes[0, idx].scatter(X_val[y_val==0, 0], X_val[y_val==0, 1], c='steelblue', s=15, alpha=0.5)
27     axes[0, idx].scatter(X_val[y_val==1, 0], X_val[y_val==1, 1], c='coral', s=15, alpha=0.5)
28     axes[0, idx].set_title(f'{name}\n({net.count_params()} params)')
29
30     axes[1, idx].plot(history['train_loss'], label='Train', alpha=0.8)
31     axes[1, idx].plot(history['val_loss'], label='Val', alpha=0.8)
32     axes[1, idx].set_xlabel('Epoch')
33     axes[1, idx].set_ylabel('Loss')
34     axes[1, idx].legend(fontsize=8)
35     axes[1, idx].set_yscale('log')
36     axes[1, idx].grid(True, alpha=0.2)
37
38 plt.suptitle('DEPTH EXPERIMENT: Deeper ≠ Better (without proper engineering)',
39             fontsize=14, fontweight='bold')
40 plt.tight_layout()
41 plt.savefig('w1_03_depth.png', dpi=100, bbox_inches='tight')
42 plt.show()
43
44 print("\n■ KEY INSIGHT: The 8-layer network likely performs WORSE than 2 layers.")
45 print("    Without BatchNorm and proper initialization, gradients vanish in deep networks.")
46 print("    We'll fix this in Week 2 with PyTorch.")

```

## ■ Week 1 Complete — Self-Assessment

### You should now be able to:

- [ ] Build a neural network from scratch (no frameworks)
- [ ] Implement forward pass AND backprop
- [ ] Explain UAT: "One wide layer CAN approximate anything, but depth is more EFFICIENT"
- [ ] Demonstrate that depth without engineering (init, norm) can hurt
- [ ] Identify overfitting by comparing train vs. val curves

### Concept Map Check:

Can you explain these connections? 1. ReLU → better gradient flow → deeper networks become trainable 2. Chain rule → backprop → efficient gradient computation 3. More parameters ≠ better generalization (will explore in Week 2) 4. Networks minimize loss ≠ networks "understand"

## ➡■ Week 2: Move to PyTorch and go DEEP properly

Open `w2_01_PyTorch_First_Network.ipynb`

# Week 2, Notebook 1: Your First PyTorch Network

## From Scratch to Framework — Feel the Difference

**What you'll build:** Rebuild your Week 1 network in PyTorch, then beat it.

**Curriculum points:**

- ⑤ Backprop → PyTorch autograd does it for you
- ⑦ Capacity ≠ performance → architecture matters
- ② ReLU → compare activations in a real framework

**Time estimate:** 45–60 minutes

### Why PyTorch?

You built everything from scratch in Week 1. Now you understand what the framework does. PyTorch's autograd computes backprop automatically — the same chain rule, just automated.

In [1]

```
1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4 import numpy as np
5 import matplotlib.pyplot as plt
6
7 torch.manual_seed(42)
8 np.random.seed(42)
9
10 # Check device
11 device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
12 print(f"Using device: {device}")
13 print(f"PyTorch version: {torch.__version__}")
```

## Part 1: Rebuild Week 1's Network in PyTorch

In [2]

```

1 # =====
2 # Generate the same moons dataset
3 # =====
4 def make_moons_torch(n=500, noise=0.15):
5     t = np.linspace(0, np.pi, n // 2)
6     x1 = np.c_[np.cos(t), np.sin(t)] + np.random.randn(n // 2, 2) * noise
7     x2 = np.c_[np.cos(t) + 0.5, -np.sin(t) + 0.5] + np.random.randn(n // 2, 2) * noise
8     X = np.vstack([x1, x2])
9     y = np.hstack([np.zeros(n // 2), np.ones(n // 2)])
10    idx = np.random.permutation(n)
11    X, y = X[idx], y[idx]
12
13    # Normalize
14    X = (X - X.mean(0)) / (X.std(0) + 1e-8)
15
16    # Convert to PyTorch tensors
17    X_tensor = torch.FloatTensor(X)
18    y_tensor = torch.FloatTensor(y).unsqueeze(1)
19    return X_tensor, y_tensor
20
21 X_all, y_all = make_moons_torch(600)
22
23 # Train/val split
24 n_train = 480
25 X_train, y_train = X_all[:n_train], y_all[:n_train]
26 X_val, y_val = X_all[n_train:], y_all[n_train:]
27
28 print(f"Train: {X_train.shape}, Val: {X_val.shape}")

```

In [3]

```

1 # =====
2 # PyTorch network – compare with your pure Python version!
3 # =====
4
5 class SimpleNet(nn.Module):
6     """Same architecture as Week 1, but in PyTorch."""
7
8     def __init__(self, hidden_size=16):
9         super().__init__()
10        self.net = nn.Sequential(
11            nn.Linear(2, hidden_size),
12            nn.ReLU(),
13            nn.Linear(hidden_size, 1),
14            nn.Sigmoid()
15        )
16
17    def forward(self, x):
18        return self.net(x)
19
20 # Compare: your pure Python version was ~50 lines of Layer class
21 # PyTorch version: 6 lines. Same result.
22 model = SimpleNet(16)
23 print(model)
24 print(f"\nTotal parameters: {sum(p.numel() for p in model.parameters())}")

```

In [4]

```

1 # =====
2 # Training loop - PyTorch handles backprop via autograd!
3 # =====
4 def train_model(model, X_train, y_train, X_val, y_val,
5                 lr=0.01, epochs=1000, optimizer_class=optim.SGD):
6     """Generic training loop."""
7     optimizer = optimizer_class(model.parameters(), lr=lr)
8     criterion = nn.MSELoss()
9
10    history = {'train_loss': [], 'val_loss': [], 'train_acc': [], 'val_acc': []}
11
12    for epoch in range(epochs):
13        # --- Training ---
14        model.train()
15        pred_train = model(X_train)
16        loss_train = criterion(pred_train, y_train)
17
18        optimizer.zero_grad()    # Clear old gradients
19        loss_train.backward()    # Compute gradients (autograd = chain rule!)
20        optimizer.step()        # Update weights
21
22        # --- Validation (no gradients needed) ---
23        model.eval()
24        with torch.no_grad():
25            pred_val = model(X_val)
26            loss_val = criterion(pred_val, y_val)
27
28            acc_train = ((pred_train > 0.5).float() == y_train).float().mean()
29            acc_val = ((pred_val > 0.5).float() == y_val).float().mean()
30
31            history['train_loss'].append(loss_train.item())
32            history['val_loss'].append(loss_val.item())
33            history['train_acc'].append(acc_train.item())
34            history['val_acc'].append(acc_val.item())
35
36            if epoch % 200 == 0:
37                print(f" Epoch {epoch:4d} | Train Loss: {loss_train.item():.4f} | "
38                      f"Val Acc: {acc_val.item():.3f}")
39
40    return history
41
42 # Train!
43 model = SimpleNet(16)
44 history = train_model(model, X_train, y_train, X_val, y_val, lr=0.05, epochs=1500)
45
46 print(f"\nFinal: Train Acc={history['train_acc'][-1]:.3f}, Val Acc={history['val_acc'][-1]:.3f}")

```

In [5]

```

1 # Visualize results
2 def plot_results(model, X_val, y_val, history, title=""):
3     fig, axes = plt.subplots(1, 3, figsize=(16, 5))
4
5     # Loss curves
6     axes[0].plot(history['train_loss'], label='Train', alpha=0.8)
7     axes[0].plot(history['val_loss'], label='Val', alpha=0.8)
8     axes[0].set_xlabel('Epoch')
9     axes[0].set_ylabel('Loss')
10    axes[0].legend()
11    axes[0].set_title('Loss')
12    axes[0].set_yscale('log')
13    axes[0].grid(True, alpha=0.2)
14
15    # Accuracy curves
16    axes[1].plot(history['train_acc'], label='Train', alpha=0.8)
17    axes[1].plot(history['val_acc'], label='Val', alpha=0.8)
18    axes[1].set_xlabel('Epoch')
19    axes[1].set_ylabel('Accuracy')
20    axes[1].legend()
21    axes[1].set_title('Accuracy')
22    axes[1].set_ylim(0.4, 1.05)
23    axes[1].grid(True, alpha=0.2)
24
25    # Decision boundary
26    xx, yy = np.meshgrid(np.linspace(-3, 3, 200), np.linspace(-3, 3, 200))
27    grid = torch.FloatTensor(np.c_[xx.ravel(), yy.ravel()])
28    model.eval()
29    with torch.no_grad():
30        zz = model(grid).numpy().reshape(xx.shape)
31
32    axes[2].contourf(xx, yy, zz, levels=20, cmap='RdYlBu_r', alpha=0.7)
33    axes[2].contour(xx, yy, zz, levels=[0.5], colors='black', linewidths=2)
34    X_np = X_val.numpy()
35    y_np = y_val.numpy().flatten()
36    axes[2].scatter(X_np[y_np==0, 0], X_np[y_np==0, 1], c='steelblue', s=25, alpha=0.6)
37    axes[2].scatter(X_np[y_np==1, 0], X_np[y_np==1, 1], c='coral', s=25, alpha=0.6)
38    axes[2].set_title('Decision Boundary')
39
40    plt.suptitle(title, fontsize=14, fontweight='bold')
41    plt.tight_layout()
42    return fig
43
44 fig = plot_results(model, X_val, y_val, history, "PyTorch SimpleNet - Moons")
45 plt.savefig('w2_01_pytorch_basic.png', dpi=100, bbox_inches='tight')
46 plt.show()

```

## Part 2: SGD vs Adam — Optimizer Comparison

Different optimizers navigate the loss landscape differently. Adam adapts learning rates per-parameter — usually converges faster.

In [6]

```

1 # =====
2 # EXPERIMENT: SGD vs Adam vs SGD+Momentum
3 # =====
4 optimizers = {
5     'SGD (lr=0.05)': (optim.SGD, {'lr': 0.05}),
6     'SGD+Momentum': (optim.SGD, {'lr': 0.05, 'momentum': 0.9}),
7     'Adam (lr=0.01)': (optim.Adam, {'lr': 0.01}),
8 }
9
10 fig, axes = plt.subplots(1, 3, figsize=(16, 5))
11 colors = ['steelblue', 'coral', 'forestgreen']
12
13 for (name, (opt_cls, opt_kwargs)), color in zip(optimizers.items(), colors):
14     torch.manual_seed(42)
15     model = SimpleNet(16)
16     optimizer = opt_cls(model.parameters(), **opt_kwargs)
17     criterion = nn.MSELoss()
18
19     losses = []
20     accs = []
21     for epoch in range(1500):
22         model.train()
23         pred = model(X_train)
24         loss = criterion(pred, y_train)
25         optimizer.zero_grad()
26         loss.backward()
27         optimizer.step()
28         losses.append(loss.item())
29
30         model.eval()
31         with torch.no_grad():
32             acc = ((model(X_val) > 0.5).float() == y_val).float().mean().item()
33             accs.append(acc)
34
35     axes[0].plot(losses, label=name, color=color, alpha=0.8)
36     axes[1].plot(accs, label=name, color=color, alpha=0.8)
37
38     # Decision boundary
39     xx, yy = np.meshgrid(np.linspace(-3, 3, 150), np.linspace(-3, 3, 150))
40     grid = torch.FloatTensor(np.c_[xx.ravel(), yy.ravel()])
41     with torch.no_grad():
42         zz = model(grid).numpy().reshape(xx.shape)
43     axes[2].contour(xx, yy, zz, levels=[0.5], colors=color, linewidths=2)
44
45 axes[0].set_title('Training Loss')
46 axes[0].set_yscale('log')
47 axes[0].legend()
48 axes[0].grid(True, alpha=0.2)
49
50 axes[1].set_title('Validation Accuracy')
51 axes[1].set_ylim(0.5, 1.05)
52 axes[1].legend()
53 axes[1].grid(True, alpha=0.2)
54
55 # Add data points to boundary plot
56 X_np = X_val.numpy()
57 y_np = y_val.numpy().flatten()
58 axes[2].scatter(X_np[y_np==0, 0], X_np[y_np==0, 1], c='steelblue', s=15, alpha=0.3)

```

```

59 axes[2].scatter(X_np[y_np==1, 0], X_np[y_np==1, 1], c='coral', s=15, alpha=0.3)
60 axes[2].set_title('Decision Boundaries (all 3)')
61 axes[2].legend(['SGD', 'SGD+Mom', 'Adam'])
62
63 plt.suptitle('OPTIMIZER COMPARISON', fontsize=14, fontweight='bold')
64 plt.tight_layout()
65 plt.savefig('w2_01_optimizers.png', dpi=100, bbox_inches='tight')
66 plt.show()
67
68 print("\n■ KEY INSIGHT: Adam usually converges faster, but all reach similar final accuracy.")
69 print("    Momentum helps SGD escape shallow local minima.")

```

## Part 3: Autograd Deep Dive — What PyTorch Does For You

Let's peek under the hood to confirm PyTorch's autograd matches our manual backprop.

In [7]

```

1  # =====
2  # Peek at autograd — see the gradients PyTorch computes
3  # =====
4  torch.manual_seed(42)
5  model = SimpleNet(4) # small for visibility
6
7  # Forward pass
8  x_sample = X_train[:1] # single sample
9  y_sample = y_train[:1]
10
11 pred = model(x_sample)
12 loss = nn.MSELoss()(pred, y_sample)
13
14 print("Forward pass:")
15 print(f"  Input: {x_sample.data}")
16 print(f"  Prediction: {pred.item():.4f}")
17 print(f"  Target: {y_sample.item():.1f}")
18 print(f"  Loss: {loss.item():.6f}")
19
20 # Backward pass
21 loss.backward()
22
23 print("\nGradients (computed by autograd = chain rule):")
24 for name, param in model.named_parameters():
25     if param.grad is not None:
26         print(f"  {name:20s} shape={list(param.shape):15s} "
27               f"grad_mean={param.grad.mean().item():+.6f}")
28
29 print("\n✓ These are EXACTLY the same gradients you computed manually in Week 1!")
30 print("  autograd = automatic chain rule application = backpropagation")

```

## ■ Self-Check

- [ ] You can write a PyTorch training loop from memory
- [ ] You understand that `loss.backward()` = your manual chain rule from Week 1
- [ ] You can compare optimizers and predict Adam will converge faster
- [ ] You can explain what `model.eval()` and `torch.no_grad()` do and why

➡■ **Next:** `w2_02_PyTorch_Going_Deeper.ipynb` — **Init, normalization, loss landscapes**

# Week 2, Notebook 2: Going Deeper — The Engineering of Deep Learning

## Initialization, Normalization, Loss Landscapes & Overparameterization

**What you'll build:** Deep networks that ACTUALLY train, plus loss landscape visualizations.

**Curriculum points:**

- ③ Overparameterization helps (and why)
- ⑥ Geometry matters — flat vs sharp minima
- ⑦ Capacity  $\neq$  performance — He init, BatchNorm, Dropout

**Time estimate:** 60 minutes (the meatiest notebook)

In [1]

```
1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4 import numpy as np
5 import matplotlib.pyplot as plt
6 from matplotlib import cm
7 from copy import deepcopy
8
9 torch.manual_seed(42)
10 np.random.seed(42)
11
12 # Generate data
13 def make_moons(n=800, noise=0.15):
14     t = np.linspace(0, np.pi, n // 2)
15     x1 = np.c_[np.cos(t), np.sin(t)] + np.random.randn(n // 2, 2) * noise
16     x2 = np.c_[np.cos(t) + 0.5, -np.sin(t) + 0.5] + np.random.randn(n // 2, 2) * noise
17     X = np.vstack([x1, x2])
18     y = np.hstack([np.zeros(n // 2), np.ones(n // 2)])
19     idx = np.random.permutation(n)
20     X = (X[idx] - X[idx].mean(0)) / (X[idx].std(0) + 1e-8)
21     return torch.FloatTensor(X), torch.FloatTensor(y[idx]).unsqueeze(1)
22
23 X_all, y_all = make_moons(800)
24 X_train, y_train = X_all[:640], y_all[:640]
25 X_val, y_val = X_all[640:], y_all[640:]
26 print(f"Data ready: {X_train.shape[0]} train, {X_val.shape[0]} val")
```

## Part 1: Initialization Matters — He vs Xavier vs Random

**Curriculum Point ⑦:** Architecture, initialization, normalization, and data quality dominate.

```

1  # =====
2  # EXPERIMENT: initialization strategies
3  # =====
4
5  def make_deep_net(depth, width, init='he'):
6      """Build a deep network with specified initialization."""
7      layers = []
8      in_dim = 2
9      for i in range(depth):
10         layer = nn.Linear(in_dim, width)
11
12         # Apply initialization
13         if init == 'he':
14             nn.init.kaiming_normal_(layer.weight, mode='fan_in', nonlinearity='relu')
15         elif init == 'xavier':
16             nn.init.xavier_normal_(layer.weight)
17         elif init == 'large_random':
18             nn.init.normal_(layer.weight, std=1.0) # Too large!
19         elif init == 'tiny_random':
20             nn.init.normal_(layer.weight, std=0.001) # Too small!
21         nn.init.zeros_(layer.bias)
22
23         layers.append(layer)
24         layers.append(nn.ReLU())
25         in_dim = width
26
27     # Output layer
28     out_layer = nn.Linear(width, 1)
29     nn.init.xavier_normal_(out_layer.weight)
30     layers.append(out_layer)
31     layers.append(nn.Sigmoid())
32
33     return nn.Sequential(*layers)
34
35
36 def quick_train(model, epochs=1000, lr=0.01):
37     optimizer = optim.Adam(model.parameters(), lr=lr)
38     criterion = nn.MSELoss()
39     losses = []
40     for epoch in range(epochs):
41         pred = model(X_train)
42         loss = criterion(pred, y_train)
43         optimizer.zero_grad()
44         loss.backward()
45         optimizer.step()
46         losses.append(loss.item())
47     return losses
48
49
50 # Compare initializations on a 6-layer network
51 depth, width = 6, 32
52 inits = ['he', 'xavier', 'large_random', 'tiny_random']
53
54 fig, axes = plt.subplots(1, 2, figsize=(14, 5))
55
56 for init_name in inits:
57     torch.manual_seed(42)
58     model = make_deep_net(depth, width, init=init_name)

```

```

59     losses = quick_train(model, epochs=1500)
60
61     n_params = sum(p.numel() for p in model.parameters())
62     axes[0].plot(losses, label=f'{init_name} ({n_params} params)', alpha=0.8)
63
64     axes[0].set_xlabel('Epoch')
65     axes[0].set_ylabel('Loss')
66     axes[0].set_title(f'Initialization Effect on {depth}-Layer Network')
67     axes[0].set_yscale('log')
68     axes[0].legend()
69     axes[0].grid(True, alpha=0.2)
70
71     # Show activation distributions for different inits
72     x_probe = X_train[:100]
73     for idx, init_name in enumerate(['he', 'large_random']):
74         torch.manual_seed(42)
75         model = make_deep_net(depth, width, init=init_name)
76
77         # Record activations at each layer
78         activations = []
79         x = x_probe
80         with torch.no_grad():
81             for layer in model:
82                 x = layer(x)
83                 if isinstance(layer, nn.ReLU):
84                     activations.append(x.numpy().flatten())
85
86         for layer_idx, act in enumerate(activations[:4]):
87             axes[1].hist(act, bins=50, alpha=0.3, density=True,
88                          label=f'{init_name} L{layer_idx}' if idx == 0 else None)
89
90     axes[1].set_title('Activation Distributions (He vs Large Random)')
91     axes[1].set_xlabel('Activation Value')
92     axes[1].legend(fontsize=8)
93     axes[1].grid(True, alpha=0.2)
94
95     plt.suptitle('INITIALIZATION: He/Xavier vs Bad Init', fontsize=14, fontweight='bold')
96     plt.tight_layout()
97     plt.savefig('w2_02_initialization.png', dpi=100, bbox_inches='tight')
98     plt.show()
99
100     print("\n■ KEY: He init keeps activations well-scaled across layers.")
101     print("    Large random → exploding activations. Tiny → dead neurons.")

```

## Part 2: BatchNorm + Dropout — Making Depth Work

BatchNorm normalizes activations between layers → stabilizes training. Dropout randomly zeroes neurons → prevents co-adaptation → reduces overfitting.

In [3]

```

1 # =====
2 # EXPERIMENT: Plain deep net vs BatchNorm vs BatchNorm+Dropout
3 # =====
4
5 class DeepNetBN(nn.Module):
6     """Deep network WITH BatchNorm and optional Dropout."""
7     def __init__(self, depth=6, width=32, dropout=0.0):
8         super().__init__()
9         layers = []
10        in_dim = 2
11        for _ in range(depth):
12            layers.append(nn.Linear(in_dim, width))
13            layers.append(nn.BatchNorm1d(width))
14            layers.append(nn.ReLU())
15            if dropout > 0:
16                layers.append(nn.Dropout(dropout))
17            in_dim = width
18        layers.append(nn.Linear(width, 1))
19        layers.append(nn.Sigmoid())
20        self.net = nn.Sequential(*layers)
21
22    def forward(self, x):
23        return self.net(x)
24
25
26 configs = {
27     'Plain (He init)': lambda: make_deep_net(8, 32, 'he'),
28     'BatchNorm': lambda: DeepNetBN(8, 32, dropout=0.0),
29     'BatchNorm + Dropout(0.3)': lambda: DeepNetBN(8, 32, dropout=0.3),
30 }
31
32 fig, axes = plt.subplots(1, 2, figsize=(14, 5))
33 colors = ['#e74c3c', '#3498db', '#2ecc71']
34
35 for (name, model_fn), color in zip(configs.items(), colors):
36     torch.manual_seed(42)
37     model = model_fn()
38     optimizer = optim.Adam(model.parameters(), lr=0.005)
39     criterion = nn.MSELoss()
40
41     train_losses, val_losses = [], []
42     for epoch in range(2000):
43         model.train()
44         pred = model(X_train)
45         loss = criterion(pred, y_train)
46         optimizer.zero_grad()
47         loss.backward()
48         optimizer.step()
49         train_losses.append(loss.item())
50
51     model.eval()
52     with torch.no_grad():
53         val_loss = criterion(model(X_val), y_val)
54         val_losses.append(val_loss.item())
55
56     axes[0].plot(train_losses, color=color, alpha=0.8, label=f'{name} (train)')
57     axes[1].plot(val_losses, color=color, alpha=0.8, label=f'{name} (val)')

```

```

59 for ax, title in zip(axes, ['Training Loss', 'Validation Loss']):
60     ax.set_xlabel('Epoch')
61     ax.set_ylabel('Loss')
62     ax.set_title(title)
63     ax.set_yscale('log')
64     ax.legend(fontsize=9)
65     ax.grid(True, alpha=0.2)
66
67 plt.suptitle('8-LAYER NETWORK: Plain vs BatchNorm vs BN+Dropout',
68             fontsize=14, fontweight='bold')
69 plt.tight_layout()
70 plt.savefig('w2_02_batchnorm.png', dpi=100, bbox_inches='tight')
71 plt.show()
72
73 print("\n■ BatchNorm stabilizes training of deep networks.")
74 print("   Dropout closes the train-val gap (reduces overfitting).")

```

## Part 3: Overparameterization — Why More Parameters Can Help

**Curriculum Point ③:** Modern networks often have MORE parameters than data points, yet generalize well.

In [4]

```

1 # =====
2 # EXPERIMENT: Overparameterization – more params, better generalization?
3 # =====
4 param_counts = []
5 val_accuracies = []
6 train_accuracies = []
7
8 widths = [4, 8, 16, 32, 64, 128, 256]
9
10 print(f"Dataset size: {X_train.shape[0]} training samples")
11 print(f"{'Width':>6} | {'Params':>8} | {'Ratio':>10} | {'Train Acc':>10} | {'Val Acc':>10}")
12 print("-" * 60)
13
14 for w in widths:
15     torch.manual_seed(42)
16     model = DeepNetBN(depth=3, width=w, dropout=0.0)
17     optimizer = optim.Adam(model.parameters(), lr=0.005)
18     criterion = nn.MSELoss()
19
20     for epoch in range(2000):
21         model.train()
22         loss = criterion(model(X_train), y_train)
23         optimizer.zero_grad()
24         loss.backward()
25         optimizer.step()
26
27     model.eval()
28     with torch.no_grad():
29         train_acc = ((model(X_train) > 0.5).float() == y_train).float().mean().item()
30         val_acc = ((model(X_val) > 0.5).float() == y_val).float().mean().item()
31
32     n_params = sum(p.numel() for p in model.parameters())
33     ratio = n_params / X_train.shape[0]
34
35     param_counts.append(n_params)
36     train_accuracies.append(train_acc)
37     val_accuracies.append(val_acc)
38
39     marker = " ← overparameterized!" if ratio > 1 else ""
40     print(f"{'w':>6} | {'n_params':>8} | {'ratio':>9.2f}x | {'train_acc':>9.3f} | {'val_acc':>9.3f}{marker}")
41
42 # Plot
43 fig, ax = plt.subplots(figsize=(10, 5))
44 ax.plot(param_counts, train_accuracies, 'o-', color='steelblue', label='Train Acc', markersize=8)
45 ax.plot(param_counts, val_accuracies, 's-', color='coral', label='Val Acc', markersize=8)
46 ax.axvline(x=X_train.shape[0], color='gray', linestyle='--', alpha=0.5, label=f'N_train={X_train.shape[0]}')
47 ax.set_xlabel('Number of Parameters')
48 ax.set_ylabel('Accuracy')
49 ax.set_title('OVERPARAMETERIZATION: More Params Can Generalize Better')
50 ax.set_xscale('log')
51 ax.legend()
52 ax.grid(True, alpha=0.2)
53 ax.set_ylim(0.7, 1.02)
54 plt.tight_layout()
55 plt.savefig('w2_02_overparameterization.png', dpi=100, bbox_inches='tight')
56 plt.show()
57
58 print("\n■ COUNTERINTUITIVE: Models with MORE params than data points can generalize BETTER.")
59
60 print("    Gradient descent + data structure = implicit regularization.")

```

## Part 4: Loss Landscape Visualization

**Curriculum Point ⑥:** Training is navigating a high-dimensional loss landscape. Flat minima → better generalization. Sharp minima → poor robustness.

In [5]

```

1 # =====
2 # Loss landscape visualization (2D slice)
3 # =====
4 # Method: take trained weights, pick 2 random directions,
5 # evaluate loss along those directions
6
7 def get_params_vector(model):
8     return torch.cat([p.data.flatten() for p in model.parameters()])
9
10 def set_params_vector(model, vec):
11     offset = 0
12     for p in model.parameters():
13         n = p.numel()
14         p.data = vec[offset:offset+n].reshape(p.shape)
15         offset += n
16
17 def compute_loss_landscape(model, X, y, resolution=30, scale=1.0):
18     """Compute loss on a 2D grid around current parameters."""
19     criterion = nn.MSELoss()
20
21     # Get current params and two random directions
22     theta = get_params_vector(model).clone()
23     d1 = torch.randn_like(theta)
24     d2 = torch.randn_like(theta)
25
26     # Normalize directions to have same norm as params
27     d1 = d1 / d1.norm() * theta.norm()
28     d2 = d2 / d2.norm() * theta.norm()
29
30     alphas = np.linspace(-scale, scale, resolution)
31     betas = np.linspace(-scale, scale, resolution)
32
33     landscape = np.zeros((resolution, resolution))
34
35     model.eval()
36     for i, a in enumerate(alphas):
37         for j, b in enumerate(betas):
38             new_params = theta + a * d1 + b * d2
39             set_params_vector(model, new_params)
40             with torch.no_grad():
41                 loss = criterion(model(X), y)
42                 landscape[j, i] = loss.item()
43
44     # Restore original params
45     set_params_vector(model, theta)
46
47     return alphas, betas, landscape
48
49
50 # Train two models: one with BatchNorm (flat minimum), one without (sharp minimum)
51 torch.manual_seed(42)
52 model_flat = DeepNetBN(depth=4, width=32, dropout=0.1)
53 optimizer = optim.Adam(model_flat.parameters(), lr=0.005)
54 criterion = nn.MSELoss()
55 for _ in range(2000):
56     model_flat.train()
57     loss = criterion(model_flat(X_train), y_train)
58     optimizer.zero_grad()

```

```

59     loss.backward()
60     optimizer.step()
61
62 torch.manual_seed(42)
63 model_sharp = make_deep_net(4, 32, 'he')
64 optimizer = optim.SGD(model_sharp.parameters(), lr=0.1)
65 for _ in range(2000):
66     loss = criterion(model_sharp(X_train), y_train)
67     optimizer.zero_grad()
68     loss.backward()
69     optimizer.step()
70
71 # Compute landscapes
72 print("Computing loss landscapes (this may take a moment)...")
73 a1, b1, L1 = compute_loss_landscape(model_flat, X_val, y_val, resolution=25, scale=0.3)
74 a2, b2, L2 = compute_loss_landscape(model_sharp, X_val, y_val, resolution=25, scale=0.3)
75
76 fig, axes = plt.subplots(1, 2, figsize=(14, 6))
77
78 for ax, alphas, betas, landscape, title in [
79     (axes[0], a1, b1, L1, 'BN+Dropout (flatter minimum)'),
80     (axes[1], a2, b2, L2, 'Plain SGD (sharper minimum)')
81 ]:
82     AA, BB = np.meshgrid(alphas, betas)
83     # Clip for visualization
84     landscape_clipped = np.clip(landscape, 0, np.percentile(landscape, 95))
85     c = ax.contourf(AA, BB, landscape_clipped, levels=30, cmap='viridis')
86     ax.contour(AA, BB, landscape_clipped, levels=15, colors='white', linewidths=0.3, alpha=0.5)
87     ax.plot(0, 0, 'r*', markersize=15, label='Trained weights')
88     ax.set_xlabel('Direction 1')
89     ax.set_ylabel('Direction 2')
90     ax.set_title(title)
91     ax.legend()
92     plt.colorbar(c, ax=ax, label='Loss')
93
94 plt.suptitle('LOSS LANDSCAPE: Flat Minima ↔ Better Generalization',
95             fontsize=14, fontweight='bold')
96 plt.tight_layout()
97 plt.savefig('w2_02_loss_landscape.png', dpi=100, bbox_inches='tight')
98 plt.show()
99
100 print("\n■ Flat minima (left) generalize better than sharp minima (right).")
101 print("   BatchNorm + proper training → flatter loss landscape.")

```

## ■ Week 2 Complete — Self-Assessment

### You should now be able to:

- [ ] Choose He init for ReLU, Xavier for sigmoid/tanh — and explain WHY
- [ ] Add BatchNorm + Dropout to any PyTorch network and predict the effect
- [ ] Explain the overparameterization paradox in simple terms
- [ ] Visualize and interpret loss landscapes
- [ ] Debug a model that isn't training (check: init? LR? architecture? data?)

### The "Karpathy Checklist" (internalize this):

- ■ Start simple (small model, small data, overfit first)
- ■ Add complexity gradually

- ■ Always compare train vs val
- ■ Visualize everything
- ■ Change ONE thing at a time

## ➡■ Week 3: Generative AI!

Open `w3_01_Pure_Autoencoder.ipynb`

# Week 3, Notebook 1: Autoencoder & VAE from Scratch

---

## Understanding Generative Models at the Lowest Level

**What you'll build:** An autoencoder and a variational autoencoder (VAE) in pure NumPy.

**Curriculum points (all integrated):**

- All 7 principles applied to generative modeling
- Plus: The reparameterization trick for VAEs

**Time estimate:** 45–60 minutes

---

### The Generative Idea

Instead of classifying data, we learn to **reconstruct** it. An autoencoder compresses data to a bottleneck (latent space), then reconstructs it. A **Variational** Autoencoder makes the latent space smooth → we can **SAMPLE** new data.

In [1]

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 np.random.seed(42)
4
5 # =====
6 # Dataset: Simple 2D patterns we can visualize
7 # =====
8 def make_dataset(n=1000):
9     """Generate a mixture of 2D Gaussians (4 clusters)."""
10    centers = np.array([[2, 2], [-2, 2], [-2, -2], [2, -2]])
11    points = []
12    labels = []
13    for i, c in enumerate(centers):
14        pts = np.random.randn(n // 4, 2) * 0.5 + c
15        points.append(pts)
16        labels.extend([i] * (n // 4))
17    return np.vstack(points).astype(np.float32), np.array(labels)
18
19 X_data, y_labels = make_dataset(1000)
20
21 plt.figure(figsize=(6, 6))
22 for i in range(4):
23     mask = y_labels == i
24     plt.scatter(X_data[mask, 0], X_data[mask, 1], s=10, alpha=0.5, label=f'Cluster {i}')
25 plt.title('Training Data: 4 Gaussian Clusters')
26 plt.legend()
27 plt.grid(True, alpha=0.2)
28 plt.axis('equal')
29 plt.savefig('w3_01_data.png', dpi=100, bbox_inches='tight')
30 plt.show()

```

## Part 1: Autoencoder — Compress and Reconstruct

In [2]

```

1 # =====
2 # Autoencoder: Encoder (2D → 1D bottleneck) → Decoder (1D → 2D)
3 # =====
4
5 class AutoEncoder:
6     """Simple autoencoder in pure NumPy."""
7
8     def __init__(self, input_dim=2, hidden_dim=8, latent_dim=1):
9         # Encoder weights
10         self.W1 = np.random.randn(input_dim, hidden_dim) * np.sqrt(2.0 / input_dim)
11         self.b1 = np.zeros(hidden_dim)
12         self.W2 = np.random.randn(hidden_dim, latent_dim) * np.sqrt(2.0 / hidden_dim)
13         self.b2 = np.zeros(latent_dim)
14
15         # Decoder weights
16         self.W3 = np.random.randn(latent_dim, hidden_dim) * np.sqrt(2.0 / latent_dim)
17         self.b3 = np.zeros(hidden_dim)
18         self.W4 = np.random.randn(hidden_dim, input_dim) * np.sqrt(2.0 / hidden_dim)
19         self.b4 = np.zeros(input_dim)
20
21     def relu(self, x):
22         return np.maximum(0, x)
23
24     def relu_grad(self, x):
25         return (x > 0).astype(float)
26
27     def encode(self, X):
28         """Compress input to latent space."""
29         self.h1 = X @ self.W1 + self.b1
30         self.a1 = self.relu(self.h1)
31         self.z = self.a1 @ self.W2 + self.b2 # Latent representation
32         return self.z
33
34     def decode(self, z):
35         """Reconstruct from latent space."""
36         self.h3 = z @ self.W3 + self.b3
37         self.a3 = self.relu(self.h3)
38         self.x_hat = self.a3 @ self.W4 + self.b4 # Reconstruction (linear output)
39         return self.x_hat
40
41     def forward(self, X):
42         self.X = X
43         z = self.encode(X)
44         x_hat = self.decode(z)
45         return x_hat
46
47     def backward(self, lr=0.001):
48         """Full backprop through encoder + decoder."""
49         m = self.X.shape[0]
50
51         # Reconstruction loss gradient: d/dx_hat (x - x_hat)^2 = -2(x - x_hat)/m
52         d_xhat = -2 * (self.X - self.x_hat) / m
53
54         # Decoder gradients
55         dW4 = self.a3.T @ d_xhat
56         db4 = d_xhat.mean(axis=0)
57         d_a3 = d_xhat @ self.W4.T
58         d_h3 = d_a3 * self.relu_grad(self.h3)

```

```

59     dw3 = self.z.T @ d_h3
60     db3 = d_h3.mean(axis=0)
61
62     # Encoder gradients (chain rule continues!)
63     d_z = d_h3 @ self.W3.T
64     dw2 = self.a1.T @ d_z
65     db2 = d_z.mean(axis=0)
66     d_a1 = d_z @ self.W2.T
67     d_h1 = d_a1 * self.relu_grad(self.h1)
68     dw1 = self.X.T @ d_h1
69     db1 = d_h1.mean(axis=0)
70
71     # Update all weights
72     for W, dw in [(self.W1, dw1), (self.W2, dw2), (self.W3, dw3), (self.W4, dw4)]:
73         W -= lr * dw
74     for b, db in [(self.b1, db1), (self.b2, db2), (self.b3, db3), (self.b4, db4)]:
75         b -= lr * db
76
77     def train(self, X, epochs=2000, lr=0.005, batch_size=64):
78         losses = []
79         for epoch in range(epochs):
80             # Mini-batch
81             idx = np.random.choice(len(X), batch_size)
82             batch = X[idx]
83
84             x_hat = self.forward(batch)
85             loss = np.mean((batch - x_hat) ** 2)
86             losses.append(loss)
87             self.backward(lr=lr)
88
89             if epoch % 500 == 0:
90                 print(f" Epoch {epoch:5d} | Reconstruction Loss: {loss:.6f}")
91         return losses
92
93     # Train the autoencoder
94     ae = AutoEncoder(input_dim=2, hidden_dim=16, latent_dim=1)
95     print("Training autoencoder (2D → 1D → 2D)...")
96     losses = ae.train(X_data, epochs=3000, lr=0.005)
97
98     # Visualize
99     fig, axes = plt.subplots(1, 3, figsize=(16, 5))
100
101     axes[0].plot(losses)
102     axes[0].set_title('Reconstruction Loss')
103     axes[0].set_xlabel('Epoch')
104     axes[0].set_yscale('log')
105     axes[0].grid(True, alpha=0.2)
106
107     x_hat = ae.forward(X_data)
108     axes[1].scatter(X_data[:, 0], X_data[:, 1], c='steelblue', s=10, alpha=0.3, label='Original')
109     axes[1].scatter(x_hat[:, 0], x_hat[:, 1], c='coral', s=10, alpha=0.3, label='Reconstructed')
110     axes[1].set_title('Original vs Reconstructed')
111     axes[1].legend()
112     axes[1].grid(True, alpha=0.2)
113     axes[1].set_aspect('equal')
114
115     z_vals = ae.encode(X_data)
116     axes[2].scatter(z_vals, np.zeros_like(z_vals), c=y_labels, s=10, cmap='tab10', alpha=0.5)

```

```
117 axes[2].set_title('Latent Space (1D)')
118 axes[2].set_xlabel('z')
119
120 plt.suptitle('AUTOENCODER: Compress 2D → 1D → Reconstruct', fontsize=14, fontweight='bold')
121 plt.tight_layout()
122 plt.savefig('w3_01_autoencoder.png', dpi=100, bbox_inches='tight')
123 plt.show()
```

## Part 2: Variational Autoencoder (VAE)

The key upgrade: instead of encoding to a POINT, encode to a DISTRIBUTION. This makes the latent space smooth → we can sample new data!

**The Reparameterization Trick:** Instead of sampling  $z \sim N(\mu, \sigma^2)$ , compute  $z = \mu + \sigma \cdot \epsilon$  where  $\epsilon \sim N(0,1)$ . This makes the sampling differentiable → backprop works!

In [3]

```

1 # =====
2 # Variational Autoencoder (VAE) – the reparameterization trick
3 # =====
4
5 class VAE:
6     """Variational Autoencoder in pure NumPy."""
7
8     def __init__(self, input_dim=2, hidden_dim=16, latent_dim=2):
9         s1 = np.sqrt(2.0 / input_dim)
10        s2 = np.sqrt(2.0 / hidden_dim)
11        s3 = np.sqrt(2.0 / latent_dim)
12
13        # Encoder → outputs mean AND log-variance
14        self.W1 = np.random.randn(input_dim, hidden_dim) * s1
15        self.b1 = np.zeros(hidden_dim)
16        self.W_mu = np.random.randn(hidden_dim, latent_dim) * s2
17        self.b_mu = np.zeros(latent_dim)
18        self.W_logvar = np.random.randn(hidden_dim, latent_dim) * s2
19        self.b_logvar = np.zeros(latent_dim)
20
21        # Decoder
22        self.W3 = np.random.randn(latent_dim, hidden_dim) * s3
23        self.b3 = np.zeros(hidden_dim)
24        self.W4 = np.random.randn(hidden_dim, input_dim) * s2
25        self.b4 = np.zeros(input_dim)
26
27        def relu(self, x): return np.maximum(0, x)
28        def relu_grad(self, x): return (x > 0).astype(float)
29
30        def encode(self, X):
31            self.h1 = X @ self.W1 + self.b1
32            self.a1 = self.relu(self.h1)
33            self.mu = self.a1 @ self.W_mu + self.b_mu
34            self.logvar = self.a1 @ self.W_logvar + self.b_logvar
35            return self.mu, self.logvar
36
37        def reparameterize(self, mu, logvar):
38            """The trick:  $z = \mu + \text{std} * \text{epsilon}$ , where  $\text{epsilon} \sim N(0,1)$ ."""
39            self.std = np.exp(0.5 * logvar)
40            self.eps = np.random.randn(*mu.shape)
41            z = mu + self.std * self.eps
42            return z
43
44        def decode(self, z):
45            self.h3 = z @ self.W3 + self.b3
46            self.a3 = self.relu(self.h3)
47            self.x_hat = self.a3 @ self.W4 + self.b4
48            return self.x_hat
49
50        def forward(self, X):
51            self.X = X
52            mu, logvar = self.encode(X)
53            self.z = self.reparameterize(mu, logvar)
54            x_hat = self.decode(self.z)
55            return x_hat, mu, logvar
56
57        def loss(self, X, x_hat, mu, logvar):
58            """VAE Loss = Reconstruction + KL Divergence."""

```

```

59     recon = np.mean((X - x_hat) ** 2)
60     kl = -0.5 * np.mean(1 + logvar - mu**2 - np.exp(logvar))
61     return recon + 0.1 * kl, recon, kl
62
63 def backward(self, lr=0.001):
64     m = self.X.shape[0]
65
66     # dL/dx_hat (reconstruction gradient)
67     d_xhat = -2 * (self.X - self.x_hat) / m
68
69     # Decoder grads
70     dw4 = self.a3.T @ d_xhat
71     db4 = d_xhat.mean(axis=0)
72     d_a3 = d_xhat @ self.W4.T
73     d_h3 = d_a3 * self.relu_grad(self.h3)
74     dw3 = self.z.T @ d_h3
75     db3 = d_h3.mean(axis=0)
76     d_z = d_h3 @ self.W3.T
77
78     # Through reparameterization: dz/dmu = 1, dz/dlogvar = 0.5*std*eps
79     d_mu = d_z + 0.1 * (2 * self.mu / m) # + KL gradient
80     d_logvar = d_z * 0.5 * self.std * self.eps + 0.1 * (-0.5 * (1 - np.exp(self.logvar)) / m)
81
82     # Encoder grads
83     dw_mu = self.a1.T @ d_mu
84     db_mu = d_mu.mean(axis=0)
85     dw_logvar = self.a1.T @ d_logvar
86     db_logvar = d_logvar.mean(axis=0)
87
88     d_a1 = d_mu @ self.W_mu.T + d_logvar @ self.W_logvar.T
89     d_h1 = d_a1 * self.relu_grad(self.h1)
90     dw1 = self.X.T @ d_h1
91     db1 = d_h1.mean(axis=0)
92
93     # Clip gradients to prevent explosions
94     for dw in [dw1, dw_mu, dw_logvar, dw3, dw4]:
95         np.clip(dw, -1.0, 1.0, out=dw)
96
97     # Update
98     for W, dw in [(self.W1, dw1), (self.W_mu, dw_mu), (self.W_logvar, dw_logvar),
99                  (self.W3, dw3), (self.W4, dw4)]:
100         W -= lr * dw
101     for b, db in [(self.b1, db1), (self.b_mu, db_mu), (self.b_logvar, db_logvar),
102                  (self.b3, db3), (self.b4, db4)]:
103         b -= lr * db
104
105 def train_loop(self, X, epochs=3000, lr=0.003, batch_size=64):
106     history = {'total': [], 'recon': [], 'kl': []}
107     for epoch in range(epochs):
108         idx = np.random.choice(len(X), batch_size)
109         batch = X[idx]
110
111         x_hat, mu, logvar = self.forward(batch)
112         total, recon, kl = self.loss(batch, x_hat, mu, logvar)
113         history['total'].append(total)
114         history['recon'].append(recon)
115         history['kl'].append(kl)

```

```

117         self.backward(lr=lr)
118
119         if epoch % 1000 == 0:
120             print(f" Epoch {epoch:5d} | Total: {total:.4f} | "
121                   f"Recon: {recon:.4f} | KL: {kl:.4f}")
122         return history
123
124
125 # Train VAE with 2D latent space
126 vae = VAE(input_dim=2, hidden_dim=32, latent_dim=2)
127 print("Training VAE (2D → 2D latent → 2D)...")
128 history = vae.train_loop(X_data, epochs=5000, lr=0.003)

```

In [4]

```

1  # =====
2  # Visualize VAE results
3  # =====
4  fig, axes = plt.subplots(2, 2, figsize=(12, 12))
5
6  # Loss curves
7  axes[0, 0].plot(history['recon'], label='Reconstruction', alpha=0.7)
8  axes[0, 0].plot(history['kl'], label='KL Divergence', alpha=0.7)
9  axes[0, 0].set_title('VAE Loss Components')
10 axes[0, 0].legend()
11 axes[0, 0].grid(True, alpha=0.2)
12
13 # Latent space
14 x_hat, mu, logvar = vae.forward(X_data)
15 z_encoded = vae.z
16 axes[0, 1].scatter(z_encoded[:, 0], z_encoded[:, 1], c=y_labels, s=10, cmap='tab10', alpha=0.5)
17 axes[0, 1].set_title('Latent Space (colored by cluster)')
18 axes[0, 1].set_xlabel('z■')
19 axes[0, 1].set_ylabel('z■')
20 axes[0, 1].grid(True, alpha=0.2)
21
22 # Reconstruction
23 axes[1, 0].scatter(X_data[:, 0], X_data[:, 1], c='steelblue', s=10, alpha=0.2, label='Original')
24 axes[1, 0].scatter(x_hat[:, 0], x_hat[:, 1], c='coral', s=10, alpha=0.2, label='Reconstructed')
25 axes[1, 0].set_title('Original vs Reconstructed')
26 axes[1, 0].legend()
27 axes[1, 0].grid(True, alpha=0.2)
28 axes[1, 0].set_aspect('equal')
29
30 # GENERATION: Sample from latent space and decode
31 z_samples = np.random.randn(500, 2) # Sample from N(0,1)
32 generated = vae.decode(z_samples)
33 axes[1, 1].scatter(X_data[:, 0], X_data[:, 1], c='steelblue', s=10, alpha=0.1, label='Real data')
34 axes[1, 1].scatter(generated[:, 0], generated[:, 1], c='red', s=10, alpha=0.3, label='GENERATED')
35 axes[1, 1].set_title('■ GENERATED Samples (from random z)')
36 axes[1, 1].legend()
37 axes[1, 1].grid(True, alpha=0.2)
38 axes[1, 1].set_aspect('equal')
39
40 plt.suptitle('VARIATIONAL AUTOENCODER – Generation from Scratch!', fontsize=14, fontweight='bold')
41 plt.tight_layout()
42 plt.savefig('w3_01_vae.png', dpi=100, bbox_inches='tight')
43 plt.show()
44
45 print("\n■ GENERATED data is in the bottom-right plot!")
46 print("    We sampled random z from N(0,1) and decoded → NEW data points!")
47 print("    This is the core idea behind generative AI.")

```

## ■ Self-Check

- ☐ You can explain: Autoencoder vs VAE → VAE adds distribution + KL divergence
- ☐ You understand the reparameterization trick:  $z = \mu + \sigma \cdot \epsilon$  makes sampling differentiable
- ☐ You GENERATED new data points from your VAE
- ☐ You can explain why KL divergence pushes the latent space toward  $N(0,1)$

➡■ **Next:** `w3_02_PyTorch_Generative_AI.ipynb` — **Full PyTorch VAE on images**

# Week 3, Notebook 2: Generative AI with PyTorch

## VAE on MNIST — Generate Handwritten Digits

**What you'll build:** A VAE that generates new handwritten digits.

**Time estimate:** 45–60 minutes

In [1]

```
1 import torch
2 import torch.nn as nn
3 import torch.nn.functional as F
4 import torch.optim as optim
5 from torch.utils.data import DataLoader, TensorDataset
6 import numpy as np
7 import matplotlib.pyplot as plt
8
9 torch.manual_seed(42)
10 device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
11 print(f"Device: {device}")
```

## Part 1: Load MNIST (built from scratch — no torchvision needed)

In [2]

```

1 # =====
2 # Generate a simple handwritten-digit-like dataset
3 # (Works without torchvision – self-contained!)
4 # =====
5 import struct
6 import os
7
8 def generate_digit_dataset(n_per_digit=200):
9     """Generate synthetic digit-like images (8x8) for demonstration.
10
11     If you have torchvision, replace this with MNIST for better results.
12     """
13     images = []
14     labels = []
15
16     for digit in range(10):
17         for _ in range(n_per_digit):
18             img = np.random.rand(8, 8) * 0.1 # background noise
19
20             # Draw digit-like patterns
21             noise = np.random.randn() * 0.3
22             if digit == 0:
23                 img[1:7, 1] = 0.8 + noise * 0.1; img[1:7, 6] = 0.8 + noise * 0.1
24                 img[1, 1:7] = 0.8 + noise * 0.1; img[6, 1:7] = 0.8 + noise * 0.1
25             elif digit == 1:
26                 img[1:7, 4] = 0.9 + noise * 0.1
27             elif digit == 2:
28                 img[1, 1:7] = 0.8; img[3, 1:7] = 0.8; img[6, 1:7] = 0.8
29                 img[1:3, 6] = 0.8; img[4:7, 1] = 0.8
30             elif digit == 3:
31                 img[1, 1:7] = 0.8; img[3, 1:7] = 0.8; img[6, 1:7] = 0.8
32                 img[1:7, 6] = 0.8
33             elif digit == 4:
34                 img[1:4, 1] = 0.8; img[3, 1:7] = 0.8; img[1:7, 6] = 0.8
35             elif digit == 5:
36                 img[1, 1:7] = 0.8; img[3, 1:7] = 0.8; img[6, 1:7] = 0.8
37                 img[1:3, 1] = 0.8; img[4:7, 6] = 0.8
38             elif digit == 6:
39                 img[1:7, 1] = 0.8; img[1, 1:7] = 0.8; img[3, 1:7] = 0.8
40                 img[6, 1:7] = 0.8; img[4:7, 6] = 0.8
41             elif digit == 7:
42                 img[1, 1:7] = 0.8; img[1:7, 6] = 0.8
43             elif digit == 8:
44                 img[1:7, 1] = 0.8; img[1:7, 6] = 0.8
45                 img[1, 1:7] = 0.8; img[3, 1:7] = 0.8; img[6, 1:7] = 0.8
46             elif digit == 9:
47                 img[1:4, 1] = 0.8; img[1:7, 6] = 0.8
48                 img[1, 1:7] = 0.8; img[3, 1:7] = 0.8
49
50             # Add random noise and slight shifts
51             img += np.random.randn(8, 8) * 0.08
52             shift_r = np.random.randint(-1, 2)
53             shift_c = np.random.randint(-1, 2)
54             img = np.roll(np.roll(img, shift_r, axis=0), shift_c, axis=1)
55             img = np.clip(img, 0, 1)
56
57             images.append(img)
58             labels.append(digit)

```

```

59     images = np.array(images, dtype=np.float32)
60     labels = np.array(labels)
61     idx = np.random.permutation(len(images))
62     return images[idx], labels[idx]
63
64
65
66 X_images, y_digits = generate_digit_dataset(300)
67 print(f"Dataset: {X_images.shape} images, {len(np.unique(y_digits))} digits")
68
69 # Show samples
70 fig, axes = plt.subplots(2, 10, figsize=(15, 3))
71 for digit in range(10):
72     mask = y_digits == digit
73     sample = X_images[mask][0]
74     axes[0, digit].imshow(sample, cmap='gray_r', vmin=0, vmax=1)
75     axes[0, digit].set_title(str(digit))
76     axes[0, digit].axis('off')
77     axes[1, digit].imshow(X_images[mask][1], cmap='gray_r', vmin=0, vmax=1)
78     axes[1, digit].axis('off')
79 plt.suptitle('Sample Training Images', fontsize=13)
80 plt.tight_layout()
81 plt.savefig('w3_02_samples.png', dpi=100, bbox_inches='tight')
82 plt.show()
83
84 # Convert to PyTorch
85 X_flat = X_images.reshape(-1, 64) # Flatten 8x8 → 64
86 X_tensor = torch.FloatTensor(X_flat).to(device)
87 y_tensor = torch.LongTensor(y_digits).to(device)
88
89 # Split
90 n_train = int(0.8 * len(X_tensor))
91 train_data = TensorDataset(X_tensor[:n_train], y_tensor[:n_train])
92 train_loader = DataLoader(train_data, batch_size=64, shuffle=True)
93 X_test = X_tensor[n_train:]
94 y_test = y_tensor[n_train:]

```

## Part 2: PyTorch VAE — Clean Implementation

In [3]

```

1 # =====
2 # Variational Autoencoder in PyTorch
3 # =====
4
5 class VAE(nn.Module):
6     def __init__(self, input_dim=64, hidden_dim=128, latent_dim=2):
7         super().__init__()
8
9         # Encoder
10        self.encoder = nn.Sequential(
11            nn.Linear(input_dim, hidden_dim),
12            nn.ReLU(),
13            nn.Linear(hidden_dim, hidden_dim // 2),
14            nn.ReLU(),
15        )
16        self.fc_mu = nn.Linear(hidden_dim // 2, latent_dim)
17        self.fc_logvar = nn.Linear(hidden_dim // 2, latent_dim)
18
19        # Decoder
20        self.decoder = nn.Sequential(
21            nn.Linear(latent_dim, hidden_dim // 2),
22            nn.ReLU(),
23            nn.Linear(hidden_dim // 2, hidden_dim),
24            nn.ReLU(),
25            nn.Linear(hidden_dim, input_dim),
26            nn.Sigmoid(), # Output in [0, 1]
27        )
28
29        def encode(self, x):
30            h = self.encoder(x)
31            return self.fc_mu(h), self.fc_logvar(h)
32
33        def reparameterize(self, mu, logvar):
34            """The reparameterization trick:  $z = \mu + \sigma \cdot \epsilon$ """
35            std = torch.exp(0.5 * logvar)
36            eps = torch.randn_like(std)
37            return mu + std * eps
38
39        def decode(self, z):
40            return self.decoder(z)
41
42        def forward(self, x):
43            mu, logvar = self.encode(x)
44            z = self.reparameterize(mu, logvar)
45            return self.decode(z), mu, logvar
46
47
48        def vae_loss(x_hat, x, mu, logvar, beta=1.0):
49            """VAE loss = Reconstruction (BCE) +  $\beta \times$  KL divergence."""
50            recon = F.mse_loss(x_hat, x, reduction='sum')
51            kl = -0.5 * torch.sum(1 + logvar - mu.pow(2) - logvar.exp())
52            return recon + beta * kl, recon, kl
53
54
55        # Create model
56        vae = VAE(input_dim=64, hidden_dim=128, latent_dim=2).to(device)
57        optimizer = optim.Adam(vae.parameters(), lr=1e-3)
58
59        print(f"VAE parameters: {sum(p.numel() for p in vae.parameters()):,}")
60        print(vae)

```

In [4]

```

1 # =====
2 # Training loop
3 # =====
4 n_epochs = 100
5 history = {'total': [], 'recon': [], 'kl': []}
6
7 for epoch in range(n_epochs):
8     vae.train()
9     epoch_loss = 0
10    epoch_recon = 0
11    epoch_kl = 0
12
13    for batch_x, batch_y in train_loader:
14        x_hat, mu, logvar = vae(batch_x)
15        loss, recon, kl = vae_loss(x_hat, batch_x, mu, logvar, beta=0.5)
16
17        optimizer.zero_grad()
18        loss.backward()
19        optimizer.step()
20
21        epoch_loss += loss.item()
22        epoch_recon += recon.item()
23        epoch_kl += kl.item()
24
25    n = len(train_loader)
26    history['total'].append(epoch_loss / n)
27    history['recon'].append(epoch_recon / n)
28    history['kl'].append(epoch_kl / n)
29
30    if epoch % 20 == 0:
31        print(f" Epoch {epoch:3d} | Total: {epoch_loss/n:.1f} | "
32              f"Recon: {epoch_recon/n:.1f} | KL: {epoch_kl/n:.1f}")
33
34    print("\n✓ Training complete!")

```

In [5]

```

1  # =====
2  # Visualize the latent space
3  # =====
4  vae.eval()
5  with torch.no_grad():
6      mu_all, _ = vae.encode(X_tensor)
7      mu_np = mu_all.cpu().numpy()
8      labels_np = y_tensor.cpu().numpy()
9
10 fig, axes = plt.subplots(1, 2, figsize=(14, 6))
11
12 # Latent space colored by digit
13 scatter = axes[0].scatter(mu_np[:, 0], mu_np[:, 1], c=labels_np, s=10,
14                           cmap='tab10', alpha=0.6)
15 axes[0].set_title('Latent Space (colored by digit)')
16 axes[0].set_xlabel('z0')
17 axes[0].set_ylabel('z1')
18 plt.colorbar(scatter, ax=axes[0], label='Digit')
19 axes[0].grid(True, alpha=0.2)
20
21 # Loss curves
22 axes[1].plot(history['recon'], label='Reconstruction', alpha=0.8)
23 axes[1].plot(history['kl'], label='KL Divergence', alpha=0.8)
24 axes[1].set_title('VAE Loss Components')
25 axes[1].set_xlabel('Epoch')
26 axes[1].legend()
27 axes[1].grid(True, alpha=0.2)
28
29 plt.tight_layout()
30 plt.savefig('w3_02_latent_space.png', dpi=100, bbox_inches='tight')
31 plt.show()

```

## Part 3: GENERATE New Digits!

In [6]

```

1 # =====
2 # Generate new images by sampling from latent space
3 # =====
4 vae.eval()
5
6 # Method 1: Random sampling from N(0,1)
7 fig, axes = plt.subplots(3, 10, figsize=(15, 5))
8
9 for row in range(3):
10     for col in range(10):
11         z = torch.randn(1, 2).to(device)
12         with torch.no_grad():
13             generated = vae.decode(z).cpu().numpy().reshape(8, 8)
14             axes[row, col].imshow(generated, cmap='gray_r', vmin=0, vmax=1)
15             axes[row, col].axis('off')
16
17 plt.suptitle('■ GENERATED Digits (sampled from N(0,1))', fontsize=14, fontweight='bold')
18 plt.tight_layout()
19 plt.savefig('w3_02_generated.png', dpi=100, bbox_inches='tight')
20 plt.show()
21
22 # Method 2: Interpolation in latent space
23 print("\nLatent Space Interpolation:")
24 fig, axes = plt.subplots(2, 10, figsize=(15, 3))
25
26 for row in range(2):
27     z_start = torch.randn(1, 2).to(device)
28     z_end = torch.randn(1, 2).to(device)
29
30     for col in range(10):
31         t = col / 9.0
32         z_interp = z_start * (1 - t) + z_end * t
33         with torch.no_grad():
34             generated = vae.decode(z_interp).cpu().numpy().reshape(8, 8)
35             axes[row, col].imshow(generated, cmap='gray_r', vmin=0, vmax=1)
36             axes[row, col].axis('off')
37
38 plt.suptitle('Latent Space Interpolation (smooth transitions)', fontsize=13)
39 plt.tight_layout()
40 plt.savefig('w3_02_interpolation.png', dpi=100, bbox_inches='tight')
41 plt.show()
42
43 print("\n✓ You just GENERATED new images with a neural network!")
44 print(" This is the foundation of all generative AI.")

```

In [7]

```

1 # =====
2 # Grid walk through latent space
3 # =====
4 n_grid = 15
5 z1 = np.linspace(-3, 3, n_grid)
6 z2 = np.linspace(-3, 3, n_grid)
7
8 canvas = np.zeros((8 * n_grid, 8 * n_grid))
9
10 vae.eval()
11 for i, z1_val in enumerate(z1):
12     for j, z2_val in enumerate(z2):
13         z = torch.FloatTensor([[z1_val, z2_val]]).to(device)
14         with torch.no_grad():
15             generated = vae.decode(z).cpu().numpy().reshape(8, 8)
16             canvas[j*8:(j+1)*8, i*8:(i+1)*8] = generated
17
18 plt.figure(figsize=(10, 10))
19 plt.imshow(canvas, cmap='gray_r', vmin=0, vmax=1)
20 plt.title('■■■ Complete Latent Space Map', fontsize=14, fontweight='bold')
21 plt.xlabel('z■')
22 plt.ylabel('z■')
23 plt.savefig('w3_02_latent_grid.png', dpi=100, bbox_inches='tight')
24 plt.show()
25
26 print("This grid shows EVERY point in latent space decoded to an image.")
27 print("Smooth transitions = the VAE learned a meaningful representation!")

```

## ■ Self-Check

- ☐ Your VAE generates recognizable digit-like patterns
- ☐ You can explain the reparameterization trick
- ☐ You see smooth interpolation in latent space
- ☐ You understand: Reconstruction loss learns content, KL divergence organizes the space

➡■ **Next:** `w3_03_PyTorch_Deploy_App.ipynb` — **Wrap your model in a web app!**

# Week 3, Notebook 3: Deploy Your Model as a Web App

## From Notebook to Full-Stack — The Finish Line

**What you'll build:** A Gradio web app where anyone can interact with your VAE.

**Time estimate:** 30 minutes

### The Deployment Stack

```
1 Your VAE Model (PyTorch)
2   ↓
3   Gradio Interface (Python)
4   ↓
5   Web App (auto-generated)
6   ↓
7   Hugging Face Spaces (free hosting)
```

## Part 1: Save Your Trained Model

In [1]

```

1 import torch
2 import torch.nn as nn
3 import torch.nn.functional as F
4 import numpy as np
5
6 # =====
7 # Recreate and train the VAE (self-contained for deployment)
8 # =====
9 class VAE(nn.Module):
10     def __init__(self, input_dim=64, hidden_dim=128, latent_dim=2):
11         super().__init__()
12         self.encoder = nn.Sequential(
13             nn.Linear(input_dim, hidden_dim), nn.ReLU(),
14             nn.Linear(hidden_dim, hidden_dim // 2), nn.ReLU(),
15         )
16         self.fc_mu = nn.Linear(hidden_dim // 2, latent_dim)
17         self.fc_logvar = nn.Linear(hidden_dim // 2, latent_dim)
18         self.decoder = nn.Sequential(
19             nn.Linear(latent_dim, hidden_dim // 2), nn.ReLU(),
20             nn.Linear(hidden_dim // 2, hidden_dim), nn.ReLU(),
21             nn.Linear(hidden_dim, input_dim), nn.Sigmoid(),
22         )
23
24     def encode(self, x):
25         h = self.encoder(x)
26         return self.fc_mu(h), self.fc_logvar(h)
27
28     def reparameterize(self, mu, logvar):
29         return mu + torch.exp(0.5 * logvar) * torch.randn_like(logvar)
30
31     def decode(self, z):
32         return self.decoder(z)
33
34     def forward(self, x):
35         mu, logvar = self.encode(x)
36         z = self.reparameterize(mu, logvar)
37         return self.decode(z), mu, logvar
38
39 # Quick train for this notebook (copy from W3_02 in practice)
40 print("Training a quick VAE for deployment demo...")
41
42 def generate_digit_dataset_quick(n_per_digit=200):
43     images, labels = [], []
44     for digit in range(10):
45         for _ in range(n_per_digit):
46             img = np.random.rand(8, 8) * 0.1
47             if digit == 0:
48                 img[1:7, 1] = 0.8; img[1:7, 6] = 0.8; img[1, 1:7] = 0.8; img[6, 1:7] = 0.8
49             elif digit == 1:
50                 img[1:7, 4] = 0.9
51             elif digit == 7:
52                 img[1, 1:7] = 0.8; img[1:7, 6] = 0.8
53             else:
54                 # Generic pattern for other digits
55                 np.random.seed(digit * 100 + _)
56                 img[1:7, 1:7] = np.random.rand(6, 6) * 0.3 + 0.3
57             img += np.random.randn(8, 8) * 0.05
58             img = np.clip(img, 0, 1)

```

```

59         images.append(img)
60         labels.append(digit)
61     return np.array(images, dtype=np.float32), np.array(labels)
62
63 X_imgs, y_labels = generate_digit_dataset_quick()
64 X_flat = torch.FloatTensor(X_imgs.reshape(-1, 64))
65
66 vae = VAE()
67 optimizer = torch.optim.Adam(vae.parameters(), lr=1e-3)
68
69 for epoch in range(80):
70     idx = torch.randperm(len(X_flat))[:64]
71     x_hat, mu, logvar = vae(X_flat[idx])
72     recon = F.mse_loss(x_hat, X_flat[idx], reduction='sum')
73     kl = -0.5 * torch.sum(1 + logvar - mu.pow(2) - logvar.exp())
74     loss = recon + 0.5 * kl
75     optimizer.zero_grad()
76     loss.backward()
77     optimizer.step()
78
79 # Save the model
80 torch.save(vae.state_dict(), 'vae_model.pth')
81 print("✓ Model saved to vae_model.pth")

```

## Part 2: Create the Gradio App

Gradio turns any Python function into a web interface. Install: `pip install gradio`

In [2]

```

1 # =====
2 # Gradio app code – save this as app.py for deployment
3 # =====
4 # The app.py file is generated below using write().
5 # It contains a full Gradio interface for the VAE.
6
7 app_lines = [
8     "import torch",
9     "import torch.nn as nn",
10    "import numpy as np",
11    "import matplotlib",
12    'matplotlib.use("Agg")',
13    "import matplotlib.pyplot as plt",
14    "import gradio as gr",
15    "",
16    "# --- VAE Model Definition ---",
17    "class VAE(nn.Module):",
18    "    def __init__(self, input_dim=64, hidden_dim=128, latent_dim=2):",
19    "        super().__init__()",
20    "        self.encoder = nn.Sequential(",
21    "            nn.Linear(input_dim, hidden_dim), nn.ReLU(),",
22    "            nn.Linear(hidden_dim, hidden_dim // 2), nn.ReLU()",
23    "        self.fc_mu = nn.Linear(hidden_dim // 2, latent_dim)",
24    "        self.fc_logvar = nn.Linear(hidden_dim // 2, latent_dim)",
25    "        self.decoder = nn.Sequential(",
26    "            nn.Linear(latent_dim, hidden_dim // 2), nn.ReLU(),",
27    "            nn.Linear(hidden_dim // 2, hidden_dim), nn.ReLU()",
28    "            nn.Linear(hidden_dim, input_dim), nn.Sigmoid()",
29    "        def decode(self, z): return self.decoder(z)",
30    "    ",
31    "model = VAE()",
32    'model.load_state_dict(torch.load("vae_model.pth", map_location="cpu"))',
33    "model.eval()",
34    "",
35    "def generate_digits(z1, z2, n_samples):",
36    "    fig, axes = plt.subplots(1, max(1,int(n_samples)), figsize=(2*n_samples,2))",
37    "    if n_samples == 1: axes = [axes]",
38    "    for i, ax in enumerate(axes):",
39    "        z = torch.FloatTensor([[z1, z2]]) if i == 0 else torch.randn(1, 2)",
40    "        with torch.no_grad(): gen = model.decode(z).numpy().reshape(8, 8)",
41    "        ax.imshow(gen, cmap='gray_r', vmin=0, vmax=1); ax.axis('off')",
42    "    plt.tight_layout()",
43    "    return fig",
44    "",
45    "demo = gr.Interface(fn=generate_digits,",
46    "    inputs=[gr.Slider(-3,3,value=0,label='z1'),",
47    "            gr.Slider(-3,3,value=0,label='z2'),",
48    "            gr.Slider(1,8,value=4,step=1,label='Samples')],",
49    "    outputs=gr.Plot(), title='VAE Digit Generator')",
50    "demo.launch()",
51 ]
52
53 with open("app.py", "w") as f:
54     f.write("\n".join(app_lines))
55
56 print("\u2713 app.py saved!")
57 print("\nTo run locally:")
58 print(" 1. pip install gradio")
59
60 print(" 2. python app.py")
61 print("\nTo deploy on Hugging Face Spaces:")
62 print(" 1. Create a new Space at huggingface.co/new-space")
63 print(" 2. Upload: app.py, vae_model.pth, requirements.txt")
64 print(" 3. Your app is live!")

```

## Part 3: Prepare for Deployment

Create the files needed for Hugging Face Spaces deployment.

In [3]

```
1 # =====
2 # Create requirements.txt and README
3 # =====
4 req_lines = ["torch", "gradio", "numpy", "matplotlib"]
5 with open("requirements.txt", "w") as f:
6     f.write("\n".join(req_lines))
7
8 readme_lines = [
9     "# VAE Digit Generator",
10    "",
11    "A Variational Autoencoder trained to generate handwritten digit patterns.",
12    "",
13    "## What This Does",
14    "- Generate: Create new digit-like images by exploring the 2D latent space",
15    "- Interpolate: See smooth transitions between different generated images",
16    "",
17    "## Built With",
18    "- PyTorch (model training)",
19    "- Gradio (web interface)",
20    "- Hugging Face Spaces (hosting)",
21    "",
22    "## Run Locally",
23    "pip install -r requirements.txt",
24    "python app.py",
25 ]
26 with open("README.md", "w") as f:
27     f.write("\n".join(readme_lines))
28
29 print("Deployment files created:")
30 print("  - app.py          (Gradio web app)")
31 print("  - vae_model.pth    (trained model weights)")
32 print("  - requirements.txt  (dependencies)")
33 print("  - README.md        (project description)")
34 print()
35 print("=" * 60)
36 print("DEPLOYMENT CHECKLIST:")
37 print("=" * 60)
38 print("1. Go to https://huggingface.co/new-space")
39 print("2. Choose Gradio as the SDK")
40 print("3. Upload all 4 files above")
41 print("4. Wait ~2 minutes for it to build")
42 print("5. Share the URL -- your app is LIVE!")
43 print()
44 print("You now have a DEPLOYED generative AI application.")
45 print("This is your portfolio piece.")
```

## ■ Congratulations — You Completed the 3-Week Plan!

### What You Built:

- **Week 1:** Neural network from scratch (pure Python) — forward pass, backprop, ReLU
- **Week 2:** Deep networks in PyTorch — initialization, BatchNorm, loss landscapes

- **Week 3:** Generative AI — VAE from scratch AND in PyTorch, deployed as a web app

## Curriculum Points Mastered:

- ■ ① Universal Approximation Theorem (width experiments)
- ■ ② ReLU changed everything (sigmoid vs ReLU comparison)
- ■ ③ Overparameterization helps (parameter scaling experiment)
- ■ ④ Neural networks minimize loss, not "understand" (built it yourself!)
- ■ ⑤ Backpropagation is just the chain rule (gradient checking proved it)
- ■ ⑥ Geometry matters (loss landscape visualization)
- ■ ⑦ Capacity  $\neq$  performance (depth + init + norm experiments)

## Your Portfolio Now Has:

- [ ] GitHub repo with all notebooks
- [ ] Deployed Gradio app on Hugging Face
- [ ] Visualizations you can explain to anyone
- [ ] Code you wrote, not just imported

## What's Next?

- **CNNs** → Apply to real image datasets (CIFAR-10, ImageNet)
- **Transformers** → The architecture behind GPT, BERT, and modern LLMs
- **Diffusion Models** → State-of-the-art image generation
- **Fine-tuning LLMs** → Hugging Face makes this accessible

*"You don't truly understand something until you can build it from scratch." You just did.*

# Week 4, Notebook 1: Graph Neural Networks from Scratch

---

## Pure Python + NumPy — Understanding Message Passing

**What you'll build:** A Graph Neural Network from absolute zero — nodes, edges, message passing, and node classification.

**New concepts:**

- Graphs as data structures for neural networks
- Adjacency matrices and degree matrices
- Message passing: aggregate neighbors, update node features
- Graph Convolutional Network (GCN) layer math

**Builds on:** Week 1–2 fundamentals (forward pass, backprop, activations)

**Time estimate:** 60 minutes

---

## Why Graphs?

Most real data isn't a grid (images) or a sequence (text). Social networks, molecules, road maps, knowledge graphs — these are all **graphs**. GNNs learn representations by passing messages between connected nodes.

## Part 1: Graphs as Matrices

In [1]

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 np.random.seed(42)
4
5 # =====
6 # A graph is just: nodes + edges
7 # We represent it as an adjacency matrix A
8 # =====
9
10 # Example: Zachary's Karate Club (simplified to 8 nodes, 2 classes)
11 # Each node is a club member; edges = friendships
12 # Task: predict which faction (0 or 1) each member belongs to
13
14 # Adjacency matrix: A[i,j] = 1 if nodes i and j are connected
15 A = np.array([
16     [0, 1, 1, 1, 0, 0, 0, 0], # Node 0
17     [1, 0, 1, 0, 0, 0, 0, 0], # Node 1
18     [1, 1, 0, 1, 1, 0, 0, 0], # Node 2
19     [1, 0, 1, 0, 0, 0, 0, 0], # Node 3
20     [0, 0, 1, 0, 0, 1, 1, 0], # Node 4
21     [0, 0, 0, 0, 1, 0, 1, 1], # Node 5
22     [0, 0, 0, 0, 1, 1, 0, 1], # Node 6
23     [0, 0, 0, 0, 0, 1, 1, 0], # Node 7
24 ], dtype=float)
25
26 # Node features: each node has a 3D feature vector
27 # (In real tasks: user profile info, atom properties, etc.)
28 X = np.random.randn(8, 3) * 0.5
29 X[:4] += np.array([1.0, 0.0, 0.5]) # Faction 0 bias
30 X[4:] += np.array([-1.0, 0.5, -0.5]) # Faction 1 bias
31
32 # Labels
33 y = np.array([0, 0, 0, 0, 1, 1, 1, 1])
34
35 print(f"Graph: {A.shape[0]} nodes, {int(A.sum() / 2)} edges")
36 print(f"Node features shape: {X.shape}")
37 print(f"Labels: {y}")
38
39 # Degree matrix: D[i,i] = number of neighbors of node i
40 D = np.diag(A.sum(axis=1))
41 print(f"\nDegree matrix diagonal: {np.diag(D)}")
42
43 # Visualize the graph
44 fig, axes = plt.subplots(1, 2, figsize=(14, 5))
45
46 # Graph structure
47 pos = {
48     0: (0, 1), 1: (1, 2), 2: (1, 0.5), 3: (0, -0.5),
49     4: (3, 0.5), 5: (4, 2), 6: (4, 0), 7: (5, 1),
50 }
51 for i in range(8):
52     for j in range(i+1, 8):
53         if A[i, j] == 1:
54             xi, yi = pos[i]
55             xj, yj = pos[j]
56             axes[0].plot([xi, xj], [yi, yj], 'gray', linewidth=1, alpha=0.5)
57
58 for i in range(8):

```

```

59     color = 'steelblue' if y[i] == 0 else 'coral'
60     xi, yi = pos[i]
61     axes[0].scatter(xi, yi, c=color, s=400, zorder=5, edgecolors='black', linewidth=1.5)
62     axes[0].text(xi, yi, str(i), ha='center', va='center', fontsize=12, fontweight='bold', color='white')
63
64 axes[0].set_title('Graph Structure (colored by class)')
65 axes[0].set_aspect('equal')
66 axes[0].axis('off')
67
68 # Adjacency matrix
69 im = axes[1].imshow(A, cmap='Blues', vmin=0, vmax=1)
70 axes[1].set_title('Adjacency Matrix A')
71 axes[1].set_xlabel('Node')
72 axes[1].set_ylabel('Node')
73 for i in range(8):
74     for j in range(8):
75         axes[1].text(j, i, int(A[i,j]), ha='center', va='center', fontsize=10)
76
77 plt.tight_layout()
78 plt.savefig('w4_01_graph.png', dpi=100, bbox_inches='tight')
79 plt.show()

```

## Part 2: The GCN Layer — Message Passing in Matrix Form

The core GCN operation is beautifully simple:

$$\mathbf{H}' = \sigma(\mathbf{D}^{-1/2} \tilde{\mathbf{A}} \mathbf{D}^{-1/2} \mathbf{H} \mathbf{W})$$

Where:

- $\tilde{\mathbf{A}} = \mathbf{A} + \mathbf{I}$  (adjacency + self-loops: every node also "messages" itself)
- $\mathbf{D}^{-1/2}$  = degree matrix of  $\tilde{\mathbf{A}}$
- $\mathbf{H}$  = node features ( $N \times F$ )
- $\mathbf{W}$  = learnable weight matrix ( $F \times F'$ )
- $\sigma$  = activation (ReLU)

In plain English: **each node averages its neighbors' features (including itself), then transforms through a weight matrix.**

In [2]

```

1 # =====
2 # GCN Layer from scratch
3 # =====
4
5 class GCNLayer:
6     """A single Graph Convolutional Network layer."""
7
8     def __init__(self, in_features, out_features):
9         # He initialization
10         self.W = np.random.randn(in_features, out_features) * np.sqrt(2.0 / in_features)
11         self.b = np.zeros(out_features)
12
13         # Gradients
14         self.dW = None
15         self.db = None
16
17         # Cache
18         self.H_in = None
19         self.Z = None
20         self.A_hat = None
21
22     def forward(self, H, A_hat):
23         """
24         H: node features (N x F_in)
25         A_hat: normalized adjacency (N x N)
26         Returns: activated output (N x F_out)
27         """
28         self.H_in = H
29         self.A_hat = A_hat
30
31         # Message passing: aggregate neighbor features
32         # AH = A_hat @ H (each node gets weighted sum of neighbor features)
33         self.AH = A_hat @ H
34
35         # Transform: apply learnable weights
36         self.Z = self.AH @ self.W + self.b
37
38         # Activate
39         self.out = np.maximum(0, self.Z) # ReLU
40         return self.out
41
42     def backward(self, d_out):
43         """Backprop through the GCN layer."""
44         # Through ReLU
45         dZ = d_out * (self.Z > 0).astype(float)
46
47         # Gradients for W and b
48         m = self.H_in.shape[0]
49         self.dW = (self.AH.T @ dZ) / m
50         self.db = np.mean(dZ, axis=0)
51
52         # Gradient for input (to pass to previous layer)
53         dAH = dZ @ self.W.T
54         dH = self.A_hat.T @ dAH # Back through message passing
55         return dH
56
57
58     def normalize_adjacency(A):

```

```

59     """Compute  $D_{\text{hat}}^{-1/2} A_{\text{hat}} D_{\text{hat}}^{-1/2}$  (symmetric normalization)."""
60     A_hat = A + np.eye(A.shape[0]) # Add self-loops
61     D_hat = np.diag(A_hat.sum(axis=1))
62     D_inv_sqrt = np.diag(1.0 / np.sqrt(np.diag(D_hat)))
63     return D_inv_sqrt @ A_hat @ D_inv_sqrt
64
65
66 # Compute normalized adjacency
67 A_norm = normalize_adjacency(A)
68
69 print("Normalized adjacency matrix (first 4x4 block):")
70 print(np.round(A_norm[:4, :4], 3))
71 print("\nNotice: diagonal is non-zero (self-loops) and rows are normalized.")
72
73 # Test a single layer
74 layer = GCNLayer(3, 4)
75 H_out = layer.forward(X, A_norm)
76 print(f"\nInput features: {X.shape}")
77 print(f"Output features: {H_out.shape}")
78 print("Each node now has features informed by its NEIGHBORS!")

```

## Part 3: Full GCN Network — Stack Layers for Classification

In [3]

```

1 # =====
2 # Full GCN for node classification
3 # =====
4
5 class GCN:
6     """2-layer Graph Convolutional Network."""
7
8     def __init__(self, in_features, hidden_features, out_classes):
9         self.layer1 = GCNLayer(in_features, hidden_features)
10        self.layer2 = GCNLayer(hidden_features, out_classes)
11
12    def softmax(self, z):
13        exp_z = np.exp(z - z.max(axis=1, keepdims=True))
14        return exp_z / exp_z.sum(axis=1, keepdims=True)
15
16    def forward(self, X, A_norm):
17        self.H1 = self.layer1.forward(X, A_norm)
18        self.Z2 = self.layer2.forward(self.H1, A_norm)
19        # Softmax for classification (override ReLU on last layer)
20        self.Z2_pre = self.layer2.Z # Pre-activation
21        self.probs = self.softmax(self.Z2_pre)
22        return self.probs
23
24    def loss(self, probs, y):
25        """Cross-entropy loss."""
26        N = len(y)
27        log_probs = -np.log(probs[np.arange(N), y] + 1e-8)
28        return np.mean(log_probs)
29
30    def backward(self, y):
31        """Backprop through the full network."""
32        N = len(y)
33
34        # Gradient of cross-entropy + softmax
35        dZ2 = self.probs.copy()
36        dZ2[np.arange(N), y] -= 1
37        dZ2 /= N
38
39        # Override the ReLU gradient for the output layer
40        # (we used softmax, not ReLU, on the output)
41        self.layer2.dW = (self.layer2.AH.T @ dZ2) / N
42        self.layer2.db = np.mean(dZ2, axis=0)
43        dAH = dZ2 @ self.layer2.W.T
44        dH1 = self.layer2.A_hat.T @ dAH
45
46        # Backprop through layer 1
47        self.layer1.backward(dH1)
48
49    def update(self, lr):
50        for layer in [self.layer1, self.layer2]:
51            layer.W -= lr * layer.dW
52            layer.b -= lr * layer.db
53
54    def train(self, X, A_norm, y, lr=0.01, epochs=200):
55        losses = []
56        accs = []
57        for epoch in range(epochs):
58            probs = self.forward(X, A_norm)

```

```

59         l = self.loss(probs, y)
60         losses.append(l)
61
62         preds = np.argmax(probs, axis=1)
63         acc = np.mean(preds == y)
64         accs.append(acc)
65
66         self.backward(y)
67         self.update(lr)
68
69         if epoch % 50 == 0:
70             print(f" Epoch {epoch:4d} | Loss: {l:.4f} | Acc: {acc:.3f}")
71     return losses, accs
72
73
74 # Train the GCN!
75 gcn = GCN(in_features=3, hidden_features=8, out_classes=2)
76 print("Training GCN on graph classification...")
77 losses, accs = gcn.train(X, A_norm, y, lr=0.05, epochs=300)
78
79 # Final predictions
80 probs = gcn.forward(X, A_norm)
81 preds = np.argmax(probs, axis=1)
82 print(f"\nFinal predictions: {preds}")
83 print(f"True labels:      {y}")
84 print(f"Accuracy:          {np.mean(preds == y):.1%}")

```

In [4]

```

1 # Visualize training and learned representations
2 fig, axes = plt.subplots(1, 3, figsize=(18, 5))
3
4 # Loss curve
5 axes[0].plot(losses, color='steelblue')
6 axes[0].set_title('Training Loss')
7 axes[0].set_xlabel('Epoch')
8 axes[0].set_ylabel('Cross-Entropy')
9 axes[0].grid(True, alpha=0.2)
10
11 # Accuracy
12 axes[1].plot(accs, color='coral')
13 axes[1].set_title('Classification Accuracy')
14 axes[1].set_xlabel('Epoch')
15 axes[1].set_ylabel('Accuracy')
16 axes[1].set_ylim(0, 1.05)
17 axes[1].grid(True, alpha=0.2)
18
19 # Learned node embeddings (hidden layer output)
20 H_hidden = gcn.layer1.forward(X, A_norm)
21 if H_hidden.shape[1] >= 2:
22     for cls in [0, 1]:
23         mask = y == cls
24         label = f'Class {cls}'
25         color = 'steelblue' if cls == 0 else 'coral'
26         axes[2].scatter(H_hidden[mask, 0], H_hidden[mask, 1],
27                        c=color, s=200, label=label, edgecolors='black', zorder=5)
28         for i in np.where(mask)[0]:
29             axes[2].annotate(str(i), (H_hidden[i, 0], H_hidden[i, 1]),
30                             ha='center', va='center', fontsize=9, fontweight='bold', color='white')
31
32 axes[2].set_title('Learned Node Embeddings (Layer 1)')
33 axes[2].legend()
34 axes[2].grid(True, alpha=0.2)
35
36 plt.suptitle('GCN FROM SCRATCH: Node Classification', fontsize=14, fontweight='bold')
37 plt.tight_layout()
38 plt.savefig('w4_01_gcn_results.png', dpi=100, bbox_inches='tight')
39 plt.show()
40
41 print("\nKEY INSIGHT: After message passing, connected nodes of the same class")
42 print("have SIMILAR embeddings. That is what GNNs learn: structural similarity.")

```

## Part 4: Why Message Passing Works — Intuition

Each GCN layer does ONE round of "talk to your neighbors":

- **Layer 1:** Each node sees its immediate neighbors (1-hop)
- **Layer 2:** Each node sees neighbors-of-neighbors (2-hop)
- **Layer K:** Each node has a K-hop receptive field

This is the graph equivalent of a CNN's receptive field growing with depth.

### Key Differences from MLPs:

|         | MLP               | GCN                  |
|---------|-------------------|----------------------|
| Input   | Fixed-size vector | Variable-size graph  |
| Sharing | No weight sharing | Same W for all nodes |

|                |                   |                                         |
|----------------|-------------------|-----------------------------------------|
| Structure      | Ignores structure | Exploits connectivity                   |
| Inductive bias | None              | Homophily (connected nodes are similar) |

## ■ Self-Check

- ☐ You can explain message passing in one sentence: "Each node aggregates its neighbors' features, transforms, and activates"
- ☐ You understand why  $A + I$  (self-loops) is needed: a node should retain its OWN features
- ☐ You can draw how 2 layers give 2-hop receptive field
- ☐ Your GCN achieves high accuracy on the toy graph

➡■ **Next:** `w4_02_PyTorch_GNN.ipynb` — **Real graphs with PyTorch Geometric**

# Week 4, Notebook 2: Graph Neural Networks with PyTorch

## From Scratch to Framework — GCN, GraphSAGE, and Real Datasets

**What you'll build:** GNNs on real graph datasets using PyTorch, exploring different aggregation strategies.

**New concepts:**

- GCN vs GraphSAGE vs GAT (different message-passing flavors)
- Node classification on real citation networks
- Graph-level pooling for graph classification
- Over-smoothing: why very deep GNNs fail

**Time estimate:** 60 minutes

In [1]

```
1 import torch
2 import torch.nn as nn
3 import torch.nn.functional as F
4 import torch.optim as optim
5 import numpy as np
6 import matplotlib.pyplot as plt
7
8 torch.manual_seed(42)
9 np.random.seed(42)
10 device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
11 print(f"Device: {device}")
```

## Part 1: GCN in Pure PyTorch (No Geometric Needed)

In [2]

```

1 # =====
2 # GCN layer in PyTorch – same math as Week 4 Notebook 1
3 # =====
4
5 class GCNConv(nn.Module):
6     """Graph Convolutional layer (Kipf & Welling, 2017)."""
7
8     def __init__(self, in_features, out_features):
9         super().__init__()
10         self.linear = nn.Linear(in_features, out_features, bias=True)
11         nn.init.kaiming_normal_(self.linear.weight, nonlinearity='relu')
12
13     def forward(self, X, A_norm):
14         """
15         X: node features (N x F_in)
16         A_norm: normalized adjacency (N x N)
17         """
18         # Message passing + transform
19         support = self.linear(X)      # Transform: X @ W + b
20         out = A_norm @ support        # Aggregate: multiply by normalized adjacency
21         return out
22
23
24 class GCNNet(nn.Module):
25     """2-layer GCN for node classification."""
26
27     def __init__(self, in_dim, hidden_dim, out_dim, dropout=0.5):
28         super().__init__()
29         self.conv1 = GCNConv(in_dim, hidden_dim)
30         self.conv2 = GCNConv(hidden_dim, out_dim)
31         self.dropout = dropout
32
33     def forward(self, X, A_norm):
34         h = self.conv1(X, A_norm)
35         h = F.relu(h)
36         h = F.dropout(h, p=self.dropout, training=self.training)
37         h = self.conv2(h, A_norm)
38         return h # raw logits; use F.log_softmax for loss
39
40
41 def normalize_adj_torch(A):
42     """Symmetric normalization:  $D^{-1/2} \hat{A} D^{-1/2}$ ."""
43     A_hat = A + torch.eye(A.size(0), device=A.device)
44     D = A_hat.sum(dim=1)
45     D_inv_sqrt = torch.diag(1.0 / torch.sqrt(D))
46     return D_inv_sqrt @ A_hat @ D_inv_sqrt
47
48 print("GCN classes defined. Ready for training!")

```

## Part 2: Generate a Larger Graph Dataset

In [3]

```

1 # =====
2 # Generate a Stochastic Block Model graph (2 communities)
3 # =====
4
5 def generate_sbm(n_nodes=200, n_classes=3, p_intra=0.15, p_inter=0.01, feat_dim=16):
6     """Generate a graph with community structure."""
7     labels = np.repeat(np.arange(n_classes), n_nodes // n_classes)
8     n = len(labels)
9
10    # Build adjacency: high prob within communities, low between
11    A = np.zeros((n, n))
12    for i in range(n):
13        for j in range(i+1, n):
14            p = p_intra if labels[i] == labels[j] else p_inter
15            if np.random.rand() < p:
16                A[i, j] = A[j, i] = 1
17
18    # Node features: noisy cluster centers
19    centers = np.random.randn(n_classes, feat_dim) * 2
20    X = np.array([centers[labels[i]] + np.random.randn(feat_dim) * 0.8 for i in range(n)])
21
22    return torch.FloatTensor(A), torch.FloatTensor(X), torch.LongTensor(labels)
23
24 A, X, y = generate_sbm(300, n_classes=3, feat_dim=16)
25 A_norm = normalize_adj_torch(A)
26
27 # Train/val/test split (random)
28 n = len(y)
29 perm = torch.randperm(n)
30 train_mask = torch.zeros(n, dtype=torch.bool)
31 val_mask = torch.zeros(n, dtype=torch.bool)
32 test_mask = torch.zeros(n, dtype=torch.bool)
33 train_mask[perm[:int(0.6*n)]] = True
34 val_mask[perm[int(0.6*n):int(0.8*n)]] = True
35 test_mask[perm[int(0.8*n):]] = True
36
37 print(f"Graph: {n} nodes, {int(A.sum()/2)} edges, {len(torch.unique(y))} classes")
38 print(f"Features per node: {X.shape[1]}")
39 print(f"Split: {train_mask.sum()} train, {val_mask.sum()} val, {test_mask.sum()} test")

```

## Part 3: Train and Evaluate

In [4]

```

1 # =====
2 # Training loop for node classification
3 # =====
4
5 model = GCNNet(in_dim=16, hidden_dim=32, out_dim=3, dropout=0.5)
6 optimizer = optim.Adam(model.parameters(), lr=0.01, weight_decay=5e-4)
7
8 history = {'train_loss': [], 'val_loss': [], 'train_acc': [], 'val_acc': []}
9
10 for epoch in range(300):
11     # Train
12     model.train()
13     logits = model(X, A_norm)
14     loss = F.cross_entropy(logits[train_mask], y[train_mask])
15
16     optimizer.zero_grad()
17     loss.backward()
18     optimizer.step()
19
20     # Eval
21     model.eval()
22     with torch.no_grad():
23         logits = model(X, A_norm)
24         train_loss = F.cross_entropy(logits[train_mask], y[train_mask]).item()
25         val_loss = F.cross_entropy(logits[val_mask], y[val_mask]).item()
26         train_acc = (logits[train_mask].argmax(1) == y[train_mask]).float().mean().item()
27         val_acc = (logits[val_mask].argmax(1) == y[val_mask]).float().mean().item()
28
29     history['train_loss'].append(train_loss)
30     history['val_loss'].append(val_loss)
31     history['train_acc'].append(train_acc)
32     history['val_acc'].append(val_acc)
33
34     if epoch % 50 == 0:
35         print(f" Epoch {epoch:3d} | Train Loss: {train_loss:.4f} | "
36               f"Val Acc: {val_acc:.3f}")
37
38 # Test
39 model.eval()
40 with torch.no_grad():
41     test_logits = model(X, A_norm)
42     test_acc = (test_logits[test_mask].argmax(1) == y[test_mask]).float().mean().item()
43     print(f"\nTest Accuracy: {test_acc:.1%}")

```

In [5]

```

1  # Visualize training and learned embeddings
2  fig, axes = plt.subplots(1, 3, figsize=(18, 5))
3
4  axes[0].plot(history['train_loss'], label='Train', alpha=0.8)
5  axes[0].plot(history['val_loss'], label='Val', alpha=0.8)
6  axes[0].set_title('Loss')
7  axes[0].legend()
8  axes[0].grid(True, alpha=0.2)
9
10 axes[1].plot(history['train_acc'], label='Train', alpha=0.8)
11 axes[1].plot(history['val_acc'], label='Val', alpha=0.8)
12 axes[1].set_title('Accuracy')
13 axes[1].set_ylim(0, 1.05)
14 axes[1].legend()
15 axes[1].grid(True, alpha=0.2)
16
17 # Node embeddings from hidden layer
18 model.eval()
19 with torch.no_grad():
20     h1 = model.conv1(X, A_norm)
21     h1 = F.relu(h1)
22     h_np = h1.numpy()
23     y_np = y.numpy()
24
25 # Simple 2D projection (first 2 dims of hidden)
26 colors = ['steelblue', 'coral', 'forestgreen']
27 for cls in range(3):
28     mask = y_np == cls
29     axes[2].scatter(h_np[mask, 0], h_np[mask, 1], c=colors[cls],
30                    s=20, alpha=0.6, label=f'Class {cls}')
31 axes[2].set_title('Learned Node Embeddings')
32 axes[2].legend()
33 axes[2].grid(True, alpha=0.2)
34
35 plt.suptitle('GCN ON COMMUNITY GRAPH', fontsize=14, fontweight='bold')
36 plt.tight_layout()
37 plt.savefig('w4_02_gcn_pytorch.png', dpi=100, bbox_inches='tight')
38 plt.show()

```

## Part 4: GraphSAGE — A Different Aggregation Strategy

GCN uses **spectral** convolution (fixed normalized adjacency). GraphSAGE uses **sampling + aggregation** — more scalable.

Key difference: instead of multiplying by the full normalized adjacency, GraphSAGE **samples** a fixed number of neighbors and aggregates with mean/max/LSTM.

In [6]

```

1 # =====
2 # GraphSAGE-style layer (mean aggregation)
3 # =====
4
5 class SAGEConv(nn.Module):
6     """Simplified GraphSAGE layer (mean aggregation)."""
7
8     def __init__(self, in_features, out_features):
9         super().__init__()
10        # Separate transforms for self and neighbors
11        self.W_self = nn.Linear(in_features, out_features, bias=False)
12        self.W_neigh = nn.Linear(in_features, out_features, bias=True)
13        nn.init.kaiming_normal_(self.W_self.weight)
14        nn.init.kaiming_normal_(self.W_neigh.weight)
15
16    def forward(self, X, A):
17        # Neighbor aggregation (mean)
18        A_hat = A + torch.eye(A.size(0), device=A.device)
19        D = A_hat.sum(dim=1, keepdim=True)
20        neigh_mean = (A_hat @ X) / D # Mean of neighbors (+ self)
21
22        # Combine self + neighbor representations
23        h_self = self.W_self(X)
24        h_neigh = self.W_neigh(neigh_mean)
25        return h_self + h_neigh
26
27
28 class SAGENet(nn.Module):
29     def __init__(self, in_dim, hidden_dim, out_dim, dropout=0.5):
30         super().__init__()
31         self.sage1 = SAGEConv(in_dim, hidden_dim)
32         self.sage2 = SAGEConv(hidden_dim, out_dim)
33         self.dropout = dropout
34
35     def forward(self, X, A):
36         h = F.relu(self.sage1(X, A))
37         h = F.dropout(h, p=self.dropout, training=self.training)
38         h = self.sage2(h, A)
39         return h
40
41
42 # Train GraphSAGE
43 sage = SAGENet(16, 32, 3)
44 optimizer = optim.Adam(sage.parameters(), lr=0.01, weight_decay=5e-4)
45
46 sage_accs = []
47 for epoch in range(300):
48     sage.train()
49     logits = sage(X, A) # Note: uses raw A, not A_norm
50     loss = F.cross_entropy(logits[train_mask], y[train_mask])
51     optimizer.zero_grad()
52     loss.backward()
53     optimizer.step()
54
55     sage.eval()
56     with torch.no_grad():
57         val_acc = (sage(X, A)[val_mask].argmax(1) == y[val_mask]).float().mean().item()
58         sage_accs.append(val_acc)

```

```
59 sage.eval()
60 with torch.no_grad():
61     test_acc_sage = (sage(X, A)[test_mask].argmax(1) == y[test_mask]).float().mean().item()
62
63
64 print(f"GraphSAGE Test Accuracy: {test_acc_sage:.1%}")
65 print(f"GCN Test Accuracy: {test_acc:.1%}")
```

## Part 5: Over-Smoothing — Why Depth Hurts in GNNs

In standard deep learning, more layers = more power. In GNNs, too many layers causes **over-smoothing**: all node representations converge to the same value.

Each layer mixes neighbor features → after K layers, every node has seen the entire graph → all embeddings become identical.

In [7]

```

1 # =====
2 # EXPERIMENT: Over-smoothing with increasing depth
3 # =====
4
5 def train_gcn_depth(depth, X, A_norm, y, train_mask, test_mask, epochs=300):
6     layers = []
7     dims = [16] + [32] * (depth - 1) + [3]
8     for i in range(len(dims) - 1):
9         layers.append(GCNConv(dims[i], dims[i+1]))
10
11     params = []
12     for l in layers:
13         params.extend(l.parameters())
14     optimizer = optim.Adam(params, lr=0.01, weight_decay=5e-4)
15
16     for epoch in range(epochs):
17         for l in layers:
18             l.train()
19         h = X
20         for i, l in enumerate(layers[:-1]):
21             h = F.relu(l(h, A_norm))
22             h = F.dropout(h, p=0.5, training=True)
23         h = layers[-1](h, A_norm)
24         loss = F.cross_entropy(h[train_mask], y[train_mask])
25         optimizer.zero_grad()
26         loss.backward()
27         optimizer.step()
28
29         for l in layers:
30             l.eval()
31         with torch.no_grad():
32             h = X
33             for i, l in enumerate(layers[:-1]):
34                 h = F.relu(l(h, A_norm))
35             h = layers[-1](h, A_norm)
36             acc = (h[test_mask].argmax(1) == y[test_mask]).float().mean().item()
37         return acc
38
39 depths = [1, 2, 3, 4, 6, 8, 12]
40 depth_accs = []
41 print("Depth experiment:")
42 for d in depths:
43     torch.manual_seed(42)
44     acc = train_gcn_depth(d, X, A_norm, y, train_mask, test_mask)
45     depth_accs.append(acc)
46     print(f" Depth {d:2d}: Test Acc = {acc:.3f}")
47
48 plt.figure(figsize=(8, 4))
49 plt.plot(depths, depth_accs, 'o-', color='steelblue', markersize=8, linewidth=2)
50 plt.xlabel('Number of GCN Layers')
51 plt.ylabel('Test Accuracy')
52 plt.title('OVER-SMOOTHING: Deeper GNNs Perform Worse')
53 plt.grid(True, alpha=0.2)
54 plt.ylim(0, 1.05)
55 plt.savefig('w4_02_oversmoothing.png', dpi=100, bbox_inches='tight')
56 plt.show()
57
58 print("\nKEY INSIGHT: 2-3 layers is the sweet spot for most GNNs.")
59
59 print("Unlike CNNs/Transformers, deeper is NOT better for graphs.")

```

## ■ Week 4 Complete — Self-Assessment

### You should now be able to:

- ☐ Build a GCN from scratch using adjacency matrix multiplication
- ☐ Explain message passing: "aggregate neighbors, transform, activate"
- ☐ Compare GCN vs GraphSAGE (spectral vs sampling-based)
- ☐ Explain over-smoothing and why 2-3 layers is typical for GNNs
- ☐ Train a GNN for node classification in PyTorch

### Key Takeaway:

GNNs generalize neural networks to **irregular structures**. The core idea — message passing — is the same as convolution, just on graphs instead of grids.

## ➡■ Week 5: Transformers!

Open `w5_01_Pure_Attention_Transformer.ipynb`

# Week 5, Notebook 1: Attention & Transformers from Scratch

## Pure Python + NumPy — The Architecture Behind GPT, BERT, and Modern AI

**What you'll build:** Self-attention, multi-head attention, and a complete Transformer encoder block — all in pure NumPy.

**New concepts:**

- Self-attention: every token attends to every other token
- Query, Key, Value decomposition
- Multi-head attention: parallel attention with different "perspectives"
- Positional encoding: giving the model a sense of order
- The Transformer block: Attention → Add & Norm → FFN → Add & Norm

**Builds on:** All previous weeks + GNN intuition (message passing  $\approx$  attention)

**Time estimate:** 60–75 minutes

### The Key Insight

A Transformer is a GNN where **every token is connected to every other token**, and the edge weights (attention) are **learned dynamically** from the data.

```
1 GNN: fixed graph structure, learned features
2 Transformer: learned graph structure (attention), learned features
```

## Part 1: Self-Attention — The Core Mechanism

In [1]

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 np.random.seed(42)
4
5 # =====
6 # Self-attention from absolute zero
7 # =====
8 # Input: a sequence of token embeddings
9 # Output: a sequence of context-aware embeddings
10 #
11 # For each token:
12 #   1. Compute Query (what am I looking for?)
13 #   2. Compute Key (what do I contain?)
14 #   3. Compute Value (what information do I carry?)
15 #   4. Attention score = Query · KeyT / sqrt(dk)
16 #   5. Output = softmax(scores) · Values
17
18 def softmax(x, axis=-1):
19     exp_x = np.exp(x - x.max(axis=axis, keepdims=True))
20     return exp_x / exp_x.sum(axis=axis, keepdims=True)
21
22
23 class SelfAttention:
24     """Single-head self-attention."""
25
26     def __init__(self, d_model, d_k):
27         """
28         d_model: input embedding dimension
29         d_k: key/query dimension (also value dimension here)
30         """
31         scale = np.sqrt(2.0 / d_model)
32         self.W_Q = np.random.randn(d_model, d_k) * scale # Query projection
33         self.W_K = np.random.randn(d_model, d_k) * scale # Key projection
34         self.W_V = np.random.randn(d_model, d_k) * scale # Value projection
35         self.d_k = d_k
36
37     def forward(self, X):
38         """
39         X: (seq_len, d_model) – sequence of token embeddings
40         Returns: (seq_len, d_k) – context-aware embeddings
41         """
42         # Project to Q, K, V
43         Q = X @ self.W_Q # (seq_len, d_k) – what each token looks for
44         K = X @ self.W_K # (seq_len, d_k) – what each token advertises
45         V = X @ self.W_V # (seq_len, d_k) – what each token carries
46
47         # Attention scores: how much does token i attend to token j?
48         scores = (Q @ K.T) / np.sqrt(self.d_k) # (seq_len, seq_len)
49
50         # Softmax: normalize to get attention weights
51         self.attn_weights = softmax(scores) # (seq_len, seq_len)
52
53         # Weighted sum of values
54         output = self.attn_weights @ V # (seq_len, d_k)
55         return output
56
57
58 # Example: a 5-word sentence

```

```

59 tokens = ["The", "cat", "sat", "on", "mat"]
60 seq_len = len(tokens)
61 d_model = 8 # embedding dimension
62 d_k = 6 # attention dimension
63
64 # Random embeddings (in practice, these come from an embedding layer)
65 X = np.random.randn(seq_len, d_model)
66
67 # Run self-attention
68 attn = SelfAttention(d_model, d_k)
69 output = attn.forward(X)
70
71 print(f"Input shape: {X.shape} (seq_len x d_model)")
72 print(f"Output shape: {output.shape} (seq_len x d_k)")
73 print(f"Attention weights shape: {attn.attn_weights.shape} (seq_len x seq_len)")
74
75 # Visualize attention weights
76 fig, ax = plt.subplots(figsize=(6, 5))
77 im = ax.imshow(attn.attn_weights, cmap='Blues', vmin=0)
78 ax.set_xticks(range(seq_len))
79 ax.set_yticks(range(seq_len))
80 ax.set_xticklabels(tokens, fontsize=12)
81 ax.set_yticklabels(tokens, fontsize=12)
82 ax.set_xlabel('Attends TO (Keys)')
83 ax.set_ylabel('Attends FROM (Queries)')
84 ax.set_title('Self-Attention Weights')
85
86 for i in range(seq_len):
87     for j in range(seq_len):
88         ax.text(j, i, f'{attn.attn_weights[i,j]:.2f}',
89                 ha='center', va='center', fontsize=9,
90                 color='white' if attn.attn_weights[i,j] > 0.3 else 'black')
91
92 plt.colorbar(im, ax=ax, label='Attention Weight')
93 plt.tight_layout()
94 plt.savefig('w5_01_attention.png', dpi=100, bbox_inches='tight')
95 plt.show()
96
97 print("\nEach row shows: how much does that token attend to every other token.")
98 print("Row sums to 1.0 (softmax). This IS the 'attention' in 'Attention is All You Need'.")

```

## Part 2: Multi-Head Attention

One attention head captures ONE type of relationship. **Multiple heads** capture different relationships in parallel.

"Head 1 might learn syntax, Head 2 might learn semantics, Head 3 might learn position."

Then we concatenate all heads and project back.

In [2]

```

1 # =====
2 # Multi-Head Attention
3 # =====
4
5 class MultiHeadAttention:
6     """Multi-head self-attention mechanism."""
7
8     def __init__(self, d_model, n_heads):
9         assert d_model % n_heads == 0, "d_model must be divisible by n_heads"
10        self.n_heads = n_heads
11        self.d_k = d_model // n_heads
12        self.d_model = d_model
13
14        # Each head has its own Q, K, V projections
15        self.heads = [SelfAttention(d_model, self.d_k) for _ in range(n_heads)]
16
17        # Output projection: concatenated heads back to d_model
18        scale = np.sqrt(2.0 / (self.d_k * n_heads))
19        self.W_O = np.random.randn(self.d_k * n_heads, d_model) * scale
20
21    def forward(self, X):
22        # Run all heads in parallel
23        head_outputs = [head.forward(X) for head in self.heads]
24
25        # Concatenate along feature dimension
26        concat = np.concatenate(head_outputs, axis=-1) # (seq_len, d_k * n_heads)
27
28        # Project back to d_model
29        output = concat @ self.W_O # (seq_len, d_model)
30        return output
31
32
33 # 4-head attention
34 mha = MultiHeadAttention(d_model=8, n_heads=4)
35 mha_output = mha.forward(X)
36
37 print(f"Multi-head attention output: {mha_output.shape}")
38 print(f"Number of heads: {mha.n_heads}, each with d_k = {mha.d_k}")
39
40 # Visualize all 4 heads
41 fig, axes = plt.subplots(1, 4, figsize=(20, 4))
42 for h in range(4):
43     weights = mha.heads[h].attn_weights
44     axes[h].imshow(weights, cmap='Blues', vmin=0, vmax=weights.max())
45     axes[h].set_title(f'Head {h+1}')
46     axes[h].set_xticks(range(seq_len))
47     axes[h].set_yticks(range(seq_len))
48     axes[h].set_xticklabels(tokens, fontsize=9, rotation=45)
49     axes[h].set_yticklabels(tokens, fontsize=9)
50
51 plt.suptitle('MULTI-HEAD ATTENTION: 4 Different Attention Patterns', fontsize=14, fontweight='bold')
52 plt.tight_layout()
53 plt.savefig('w5_01_multihead.png', dpi=100, bbox_inches='tight')
54 plt.show()
55
56 print("\nEach head learns a DIFFERENT attention pattern!")
57 print("This is like having 4 different 'perspectives' on the same data.")

```

## Part 3: The Complete Transformer Block

A full Transformer encoder block:

```
1 Input → Multi-Head Attention → Add & Norm → FFN → Add & Norm → Output
2           ↑_____|                               ↑_____|
3           (residual connection)                     (residual connection)
```

In [3]

```

1 # =====
2 # Layer Normalization
3 # =====
4
5 class LayerNorm:
6     """Layer normalization (Ba et al., 2016)."""
7
8     def __init__(self, d_model, eps=1e-6):
9         self.gamma = np.ones(d_model)
10        self.beta = np.zeros(d_model)
11        self.eps = eps
12
13    def forward(self, x):
14        mean = x.mean(axis=-1, keepdims=True)
15        std = x.std(axis=-1, keepdims=True)
16        return self.gamma * (x - mean) / (std + self.eps) + self.beta
17
18
19 # =====
20 # Feed-Forward Network (position-wise)
21 # =====
22
23 class FeedForward:
24     """Position-wise feed-forward network."""
25
26     def __init__(self, d_model, d_ff):
27         scale1 = np.sqrt(2.0 / d_model)
28         scale2 = np.sqrt(2.0 / d_ff)
29         self.W1 = np.random.randn(d_model, d_ff) * scale1
30         self.b1 = np.zeros(d_ff)
31         self.W2 = np.random.randn(d_ff, d_model) * scale2
32         self.b2 = np.zeros(d_model)
33
34    def forward(self, x):
35        h = np.maximum(0, x @ self.W1 + self.b1) # ReLU
36        return h @ self.W2 + self.b2
37
38
39 # =====
40 # Positional Encoding
41 # =====
42
43 def positional_encoding(seq_len, d_model):
44     """Sinusoidal positional encoding (Vaswani et al., 2017)."""
45     PE = np.zeros((seq_len, d_model))
46     for pos in range(seq_len):
47         for i in range(0, d_model, 2):
48             PE[pos, i] = np.sin(pos / 10000 ** (i / d_model))
49             if i + 1 < d_model:
50                 PE[pos, i+1] = np.cos(pos / 10000 ** (i / d_model))
51     return PE
52
53
54 # =====
55 # Complete Transformer Encoder Block
56 # =====
57
58 class TransformerBlock:

```

```

59     """One Transformer encoder block."""
60
61     def __init__(self, d_model, n_heads, d_ff):
62         self.mha = MultiHeadAttention(d_model, n_heads)
63         self.norm1 = LayerNorm(d_model)
64         self.ff = FeedForward(d_model, d_ff)
65         self.norm2 = LayerNorm(d_model)
66
67     def forward(self, X):
68         # Multi-head attention + residual + norm
69         attn_out = self.mha.forward(X)
70         X = self.norm1.forward(X + attn_out) # Residual connection
71
72         # Feed-forward + residual + norm
73         ff_out = self.ff.forward(X)
74         X = self.norm2.forward(X + ff_out) # Residual connection
75
76         return X
77
78
79 # Build a 2-block Transformer encoder
80 d_model = 16
81 n_heads = 4
82 d_ff = 64
83 seq_len = 8
84
85 # Random input embeddings + positional encoding
86 X_seq = np.random.randn(seq_len, d_model) * 0.5
87 PE = positional_encoding(seq_len, d_model)
88 X_input = X_seq + PE
89
90 # Stack 2 Transformer blocks
91 block1 = TransformerBlock(d_model, n_heads, d_ff)
92 block2 = TransformerBlock(d_model, n_heads, d_ff)
93
94 h = block1.forward(X_input)
95 h = block2.forward(h)
96
97 print(f"Input: {X_input.shape} (seq_len x d_model)")
98 print(f"Output: {h.shape} (seq_len x d_model)")
99 print(f"\nTransformer block components:")
100 print(f"  Multi-Head Attention: {n_heads} heads, d_k = {d_model // n_heads}")
101 print(f"  Feed-Forward: {d_model} -> {d_ff} -> {d_model}")
102 print(f"  Positional Encoding: sinusoidal")
103 print(f"\nTotal blocks: 2 (each token has seen the full sequence TWICE)")

```

In [4]

```

1 # Visualize positional encoding
2 PE_vis = positional_encoding(50, 32)
3
4 fig, axes = plt.subplots(1, 2, figsize=(14, 5))
5
6 axes[0].imshow(PE_vis.T, cmap='RdBu_r', aspect='auto')
7 axes[0].set_xlabel('Position')
8 axes[0].set_ylabel('Dimension')
9 axes[0].set_title('Positional Encoding Matrix')
10
11 for dim in [0, 1, 2, 3, 6, 10]:
12     axes[1].plot(PE_vis[:, dim], label=f'dim {dim}', alpha=0.8)
13 axes[1].set_xlabel('Position')
14 axes[1].set_ylabel('Value')
15 axes[1].set_title('PE by Dimension (sinusoidal waves)')
16 axes[1].legend(fontsize=8)
17 axes[1].grid(True, alpha=0.2)
18
19 plt.suptitle('POSITIONAL ENCODING: Giving Order to the Transformer', fontsize=14, fontweight='bold')
20 plt.tight_layout()
21 plt.savefig('w5_01_positional.png', dpi=100, bbox_inches='tight')
22 plt.show()
23
24 print("Lower dimensions = high frequency (position changes fast)")
25 print("Higher dimensions = low frequency (captures longer-range position)")
26 print("This lets the model learn both local and global position relationships.")

```

## ■ Self-Check

- [ ] You can explain attention in one sentence: "Each token computes a weighted average of all tokens' values, weighted by query-key similarity"
- [ ] You understand Q, K, V: Query = "what I want", Key = "what I offer", Value = "what I carry"
- [ ] You can explain multi-head: "parallel attention heads capture different relationship types"
- [ ] You know why positional encoding exists: attention is permutation-invariant, PE breaks symmetry
- [ ] You see the connection: GNN message passing is attention with a fixed graph; Transformers learn the graph

➡■ **Next:** `w5_02_PyTorch_Transformer.ipynb` — **Build a real Transformer in PyTorch**

# Week 5, Notebook 2: Transformers with PyTorch

## Build, Train, and Use Transformer Models

**What you'll build:** A Transformer-based sequence classifier and explore pre-trained models with Hugging Face.

**New concepts:**

- `nn.TransformerEncoder` in PyTorch
- Causal masking for autoregressive models
- Transfer learning: using pre-trained Transformers
- Tokenization and the Hugging Face ecosystem

**Time estimate:** 60 minutes

In [1]

```
1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4 import torch.nn.functional as F
5 import numpy as np
6 import matplotlib.pyplot as plt
7 import math
8
9 torch.manual_seed(42)
10 np.random.seed(42)
11 device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
12 print(f"Device: {device}")
```

## Part 1: Transformer Encoder for Sequence Classification

In [2]

```

1 # =====
2 # Transformer encoder for classifying sequences
3 # Task: classify sequences of numbers as "ascending" or "not"
4 # =====
5
6 def generate_sequence_data(n_samples=2000, seq_len=8, vocab_size=50):
7     """Generate sequences and labels: 1=ascending, 0=random."""
8     x = []
9     y = []
10    for _ in range(n_samples):
11        if np.random.rand() > 0.5:
12            # Ascending sequence (with noise)
13            start = np.random.randint(0, vocab_size - seq_len - 5)
14            seq = np.sort(np.random.choice(range(start, start + seq_len + 5),
15                                         size=seq_len, replace=False))
16            x.append(seq)
17            y.append(1)
18        else:
19            # Random sequence
20            seq = np.random.randint(0, vocab_size, size=seq_len)
21            x.append(seq)
22            y.append(0)
23
24    x = torch.LongTensor(np.array(x))
25    y = torch.LongTensor(y)
26    return x, y
27
28
29 x_data, y_data = generate_sequence_data(3000)
30 n_train = 2400
31 x_train, y_train = x_data[:n_train], y_data[:n_train]
32 x_val, y_val = x_data[n_train:], y_data[n_train:]
33
34 print(f"Training: {x_train.shape}, Validation: {x_val.shape}")
35 print(f"Sample ascending: {x_data[y_data == 1][0].numpy()}")
36 print(f"Sample random: {x_data[y_data == 0][0].numpy()}")

```

In [3]

```

1 # =====
2 # Transformer Classifier
3 # =====
4
5 class PositionalEncoding(nn.Module):
6     """Sinusoidal positional encoding."""
7     def __init__(self, d_model, max_len=512):
8         super().__init__()
9         pe = torch.zeros(max_len, d_model)
10        position = torch.arange(0, max_len, dtype=torch.float).unsqueeze(1)
11        div_term = torch.exp(torch.arange(0, d_model, 2).float() * (-math.log(10000.0) / d_model))
12        pe[:, 0::2] = torch.sin(position * div_term)
13        pe[:, 1::2] = torch.cos(position * div_term)
14        self.register_buffer('pe', pe.unsqueeze(0)) # (1, max_len, d_model)
15
16    def forward(self, x):
17        return x + self.pe[:, :x.size(1), :]
18
19
20 class TransformerClassifier(nn.Module):
21     """Transformer encoder for sequence classification."""
22
23    def __init__(self, vocab_size, d_model=64, n_heads=4, n_layers=2,
24                 d_ff=128, n_classes=2, max_len=100, dropout=0.1):
25        super().__init__()
26
27        self.embedding = nn.Embedding(vocab_size, d_model)
28        self.pos_encoder = PositionalEncoding(d_model, max_len)
29        self.dropout = nn.Dropout(dropout)
30
31        # Transformer encoder layers
32        encoder_layer = nn.TransformerEncoderLayer(
33            d_model=d_model, nhead=n_heads, dim_feedforward=d_ff,
34            dropout=dropout, batch_first=True
35        )
36        self.transformer = nn.TransformerEncoder(encoder_layer, num_layers=n_layers)
37
38        # Classification head
39        self.classifier = nn.Sequential(
40            nn.Linear(d_model, d_model // 2),
41            nn.ReLU(),
42            nn.Dropout(dropout),
43            nn.Linear(d_model // 2, n_classes)
44        )
45
46    def forward(self, x):
47        # Embed tokens + add position
48        h = self.embedding(x) * math.sqrt(64)
49        h = self.pos_encoder(h)
50        h = self.dropout(h)
51
52        # Transformer encoder
53        h = self.transformer(h)
54
55        # Pool: mean of all token representations
56        h = h.mean(dim=1) # (batch, d_model)
57
58        # Classify

```

```

59         return self.classifier(h)
60
61
62 model = TransformerClassifier(vocab_size=50, d_model=64, n_heads=4,
63                             n_layers=2, d_ff=128, n_classes=2)
64 print(model)
65 print(f"\nTotal parameters: {sum(p.numel() for p in model.parameters()):,}")

```

In [4]

```

1  # =====
2  # Training loop
3  # =====
4  optimizer = optim.Adam(model.parameters(), lr=0.001)
5  criterion = nn.CrossEntropyLoss()
6
7  history = {'train_loss': [], 'val_loss': [], 'train_acc': [], 'val_acc': []}
8  batch_size = 64
9
10 for epoch in range(60):
11     model.train()
12     epoch_loss = 0
13     correct = 0
14     total = 0
15
16     # Mini-batch training
17     perm = torch.randperm(n_train)
18     for i in range(0, n_train, batch_size):
19         idx = perm[i:i+batch_size]
20         xb, yb = X_train[idx], y_train[idx]
21
22         logits = model(xb)
23         loss = criterion(logits, yb)
24
25         optimizer.zero_grad()
26         loss.backward()
27         optimizer.step()
28
29         epoch_loss += loss.item() * len(xb)
30         correct += (logits.argmax(1) == yb).sum().item()
31         total += len(xb)
32
33     # Validation
34     model.eval()
35     with torch.no_grad():
36         val_logits = model(X_val)
37         val_loss = criterion(val_logits, y_val).item()
38         val_acc = (val_logits.argmax(1) == y_val).float().mean().item()
39         train_acc = correct / total
40
41     history['train_loss'].append(epoch_loss / total)
42     history['val_loss'].append(val_loss)
43     history['train_acc'].append(train_acc)
44     history['val_acc'].append(val_acc)
45
46     if epoch % 10 == 0:
47         print(f" Epoch {epoch:3d} | Train Acc: {train_acc:.3f} | Val Acc: {val_acc:.3f}")
48
49 print(f"\nFinal Val Accuracy: {history['val_acc'][-1]:.1%}")

```

In [5]

```

1  # Visualize training
2  fig, axes = plt.subplots(1, 2, figsize=(12, 5))
3
4  axes[0].plot(history['train_loss'], label='Train', alpha=0.8)
5  axes[0].plot(history['val_loss'], label='Val', alpha=0.8)
6  axes[0].set_title('Loss')
7  axes[0].legend()
8  axes[0].grid(True, alpha=0.2)
9
10 axes[1].plot(history['train_acc'], label='Train', alpha=0.8)
11 axes[1].plot(history['val_acc'], label='Val', alpha=0.8)
12 axes[1].set_title('Accuracy')
13 axes[1].set_ylim(0.4, 1.05)
14 axes[1].legend()
15 axes[1].grid(True, alpha=0.2)
16
17 plt.suptitle('TRANSFORMER CLASSIFIER: Sequence Classification', fontsize=14, fontweight='bold')
18 plt.tight_layout()
19 plt.savefig('w5_02_training.png', dpi=100, bbox_inches='tight')
20 plt.show()

```

## Part 2: Attention Visualization — What Does the Transformer See?

In [6]

```

1 # =====
2 # Extract and visualize attention weights
3 # =====
4 # We need to hook into the transformer to get attention
5
6 model.eval()
7 sample_idx = 0
8 x_sample = X_val[sample_idx:sample_idx+1]
9 y_true = y_val[sample_idx].item()
10
11 # Get prediction
12 with torch.no_grad():
13     logit = model(x_sample)
14     pred = logit.argmax(1).item()
15
16 print(f"Sequence: {x_sample[0].numpy()}")
17 print(f"True label: {'ascending' if y_true else 'random'}")
18 print(f"Prediction: {'ascending' if pred else 'random'}")
19
20 # For visualization, compute attention manually
21 with torch.no_grad():
22     h = model.embedding(x_sample) * math.sqrt(64)
23     h = model.pos_encoder(h)
24
25     # Get first layer's attention
26     layer0 = model.transformer.layers[0]
27     # Use the self_attn module directly
28     attn_out, attn_weights = layer0.self_attn(h, h, h, need_weights=True)
29
30 attn_np = attn_weights[0].numpy() # (seq_len, seq_len)
31 tokens = [str(t) for t in x_sample[0].numpy()]
32
33 fig, ax = plt.subplots(figsize=(7, 6))
34 im = ax.imshow(attn_np, cmap='Blues')
35 ax.set_xticks(range(len(tokens)))
36 ax.set_yticks(range(len(tokens)))
37 ax.set_xticklabels(tokens, fontsize=10)
38 ax.set_yticklabels(tokens, fontsize=10)
39 ax.set_xlabel('Keys')
40 ax.set_ylabel('Queries')
41 label = 'ascending' if y_true else 'random'
42 ax.set_title(f'Attention Weights (Layer 1) - "{label}" sequence')
43
44 for i in range(len(tokens)):
45     for j in range(len(tokens)):
46         ax.text(j, i, f'{attn_np[i,j]:.2f}', ha='center', va='center',
47                 fontsize=8, color='white' if attn_np[i,j] > 0.2 else 'black')
48
49 plt.colorbar(im, ax=ax)
50 plt.tight_layout()
51 plt.savefig('w5_02_attention_vis.png', dpi=100, bbox_inches='tight')
52 plt.show()

```

## Part 3: Causal (Autoregressive) Masking — How GPT Works

GPT-style models can only attend to **past** tokens (not future ones). This is enforced with a **causal mask**: an upper-triangular matrix of -infinity.

In [7]

```

1 # =====
2 # Causal masking demonstration
3 # =====
4
5 seq_len = 6
6 tokens_demo = ["I", "love", "deep", "learn", "-ing", "!"]
7
8 # Causal mask: position i can only attend to positions <= i
9 causal_mask = torch.triu(torch.ones(seq_len, seq_len) * float('-inf'), diagonal=1)
10
11 print("Causal mask (GPT-style):")
12 print("0 = can attend, -inf = blocked")
13 for i in range(seq_len):
14     row = []
15     for j in range(seq_len):
16         row.append(" 0 " if causal_mask[i,j] == 0 else " -inf")
17     print(f" {tokens_demo[i]:6s}: {' '.join(row)}")
18
19 # Visualize
20 fig, axes = plt.subplots(1, 2, figsize=(12, 5))
21
22 # Bidirectional (BERT-style)
23 bidir = np.ones((seq_len, seq_len))
24 axes[0].imshow(bidir, cmap='Blues', vmin=0, vmax=1)
25 axes[0].set_title('Bidirectional Attention (BERT)')
26 axes[0].set_xticks(range(seq_len))
27 axes[0].set_yticks(range(seq_len))
28 axes[0].set_xticklabels(tokens_demo, fontsize=9)
29 axes[0].set_yticklabels(tokens_demo, fontsize=9)
30
31 # Causal (GPT-style)
32 causal_vis = np.tril(np.ones((seq_len, seq_len)))
33 axes[1].imshow(causal_vis, cmap='Blues', vmin=0, vmax=1)
34 axes[1].set_title('Causal Attention (GPT)')
35 axes[1].set_xticks(range(seq_len))
36 axes[1].set_yticks(range(seq_len))
37 axes[1].set_xticklabels(tokens_demo, fontsize=9)
38 axes[1].set_yticklabels(tokens_demo, fontsize=9)
39
40 plt.suptitle('BERT vs GPT: Attention Masks', fontsize=14, fontweight='bold')
41 plt.tight_layout()
42 plt.savefig('w5_02_causal.png', dpi=100, bbox_inches='tight')
43 plt.show()
44
45 print("\nBERT: every token sees ALL other tokens (bidirectional)")
46 print("GPT: each token only sees PAST tokens (causal/autoregressive)")
47 print("This is the key architectural difference between BERT and GPT.")

```

## Part 4: The Transformer Landscape — What's Out There

| Model       | Type            | Attention     | Key Use Case                        |
|-------------|-----------------|---------------|-------------------------------------|
| <b>BERT</b> | Encoder         | Bidirectional | Understanding (classification, NER) |
| <b>GPT</b>  | Decoder         | Causal        | Generation (text, code)             |
| <b>T5</b>   | Encoder-Decoder | Both          | Any text-to-text task               |
| <b>ViT</b>  | Encoder         | Bidirectional | Image classification                |

|         |                 |             |                     |
|---------|-----------------|-------------|---------------------|
| CLIP    | Dual Encoder    | Cross-modal | Image-text matching |
| Whisper | Encoder-Decoder | Both        | Speech recognition  |

The Transformer architecture is now the foundation of virtually all state-of-the-art models in NLP, vision, speech, and multimodal AI.

## ■ Week 5 Complete — Self-Assessment

### You should now be able to:

- ☐ Build a Transformer encoder from scratch (pure Python AND PyTorch)
- ☐ Explain self-attention: "each token attends to all others via Q, K, V"
- ☐ Explain multi-head attention: "parallel attention for different relationships"
- ☐ Explain positional encoding: "sinusoidal signal that breaks permutation invariance"
- ☐ Distinguish BERT (bidirectional) from GPT (causal)
- ☐ Train a Transformer classifier in PyTorch
- ☐ Visualize attention weights to understand what the model learns

## ■ Congratulations — You Completed All 5 Weeks!

### Your Learning Journey:

- **Week 1:** Neural networks from scratch — forward pass, backprop, ReLU
- **Week 2:** Deep networks in PyTorch — init, normalization, loss landscapes
- **Week 3:** Generative AI — VAEs, latent spaces, deployment
- **Week 4:** Graph Neural Networks — message passing, GCN, GraphSAGE
- **Week 5:** Transformers — attention, multi-head, positional encoding, GPT vs BERT

### What's Next?

- **Fine-tune a pre-trained model** — Hugging Face makes this 10 lines of code
- **Diffusion Models** — the architecture behind Stable Diffusion, DALL-E
- **Reinforcement Learning from Human Feedback (RLHF)** — how ChatGPT is trained
- **Build something real** — take a problem you care about and apply what you've learned

*"You've built every major architecture from scratch. Nothing in deep learning is magic anymore."*

# Deep Learning Cheat Sheet

## Every Key Pattern from the 3-Week Course — One Notebook

Use this as your go-to reference after completing the course. Each section is self-contained — copy-paste what you need.

### 1. The Single Neuron (Pure Python)

```
1 output = activation(weights · inputs + bias)
```

This is the atom of deep learning. Everything else is layers of this.

In [1]

```
1 import numpy as np
2
3 # === SINGLE NEURON ===
4 def neuron(x, w, b, activation='relu'):
5     z = np.dot(x, w) + b
6     if activation == 'relu':
7         return np.maximum(0, z)
8     elif activation == 'sigmoid':
9         return 1.0 / (1.0 + np.exp(-np.clip(z, -500, 500)))
10    return z # linear
11
12 # Example
13 x = np.array([1.0, 0.5])
14 w = np.random.randn(2) * 0.1
15 b = 0.0
16 print(f"ReLU output: {neuron(x, w, b, 'relu'):.4f}")
17 print(f"Sigmoid output: {neuron(x, w, b, 'sigmoid'):.4f}")
```

### 2. Activation Functions — When to Use What

| Activation | Use When                   | Watch Out For                    |
|------------|----------------------------|----------------------------------|
| ReLU       | Default for hidden layers  | Dead neurons (output always 0)   |
| Sigmoid    | Binary output (last layer) | Vanishing gradients in deep nets |
| Tanh       | Centered output needed     | Same vanishing gradient problem  |
| Linear     | Regression output          | No non-linearity added           |

**Rule of thumb:** ReLU for hidden layers, sigmoid for binary classification output, linear for regression output.

In [2]

```
1 # === ACTIVATION FUNCTIONS AND THEIR DERIVATIVES ===
2
3 def relu(z):      return np.maximum(0, z)
4 def relu_d(z):    return (z > 0).astype(float)
5
6 def sigmoid(z):
7     s = 1.0 / (1.0 + np.exp(-np.clip(z, -500, 500)))
8     return s
9 def sigmoid_d(z):
10    s = sigmoid(z)
11    return s * (1 - s)
12
13 # KEY INSIGHT: Sigmoid max gradient = 0.25 (vanishes when chained)
14 #               ReLU gradient = 1 when active (never vanishes!)
15 z_test = np.array([0.0])
16 print(f"Sigmoid gradient at z=0: {sigmoid_d(z_test)[0]:.2f} (max possible)")
17 print(f"ReLU gradient at z=0.1: {relu_d(np.array([0.1]))[0]:.0f} (always 1 when active)")
```

### 3. Backpropagation — The Chain Rule Pattern

```
1 Forward:  $x \rightarrow z=Wx+b \rightarrow a=\text{act}(z) \rightarrow \text{loss}$ 
2 Backward:  $dL/dW = dL/da \cdot da/dz \cdot dz/dW$ 
```

Every backward pass follows this pattern. It's dynamic programming for gradients.

In [3]

```
1 # === BACKPROP IN 10 LINES ===
2 # Given: layer with weights W, bias b, activation
3
4 def forward(x, W, b):
5     z = x @ W + b
6     a = relu(z)
7     return z, a
8
9 def backward(x, z, da, W):
10    dz = da * relu_d(z)          # gradient through activation
11    dW = (x.T @ dz) / len(x)     # gradient for weights
12    db = np.mean(dz, axis=0)     # gradient for bias
13    dx = dz @ W.T               # gradient for input (pass to prev layer)
14    return dW, db, dx
15
16 # Update rule:  $W -= \text{learning\_rate} * dW$ 
17 print("Backprop is just the chain rule applied layer by layer. That's it.")
```

### 4. Initialization — He vs Xavier

| Init Method | Formula                                | Use With         |
|-------------|----------------------------------------|------------------|
| He          | $\text{std} = \sqrt{2/\text{fan\_in}}$ | ReLU activations |

|               |                                   |                          |
|---------------|-----------------------------------|--------------------------|
| <b>Xavier</b> | <code>std = sqrt(1/fan_in)</code> | Sigmoid/Tanh activations |
| <b>Zeros</b>  | Never for weights!                | Only for biases          |

Wrong init = network won't train. This is the #1 silent killer.

In [4]

```

1  # === INITIALIZATION PATTERNS ===
2
3  # Pure Python
4  def he_init(fan_in, fan_out):
5      return np.random.randn(fan_in, fan_out) * np.sqrt(2.0 / fan_in)
6
7  def xavier_init(fan_in, fan_out):
8      return np.random.randn(fan_in, fan_out) * np.sqrt(1.0 / fan_in)
9
10 # PyTorch equivalents
11 import torch
12 import torch.nn as nn
13
14 layer = nn.Linear(64, 32)
15 nn.init.kaiming_normal_(layer.weight, mode='fan_in', nonlinearity='relu') # He
16 # nn.init.xavier_normal_(layer.weight) # Xavier
17
18 print(f"He init std for fan_in=64: {np.sqrt(2.0/64):.4f}")
19 print(f"Xavier std for fan_in=64: {np.sqrt(1.0/64):.4f}")

```

## 5. PyTorch Training Loop — The Template

This is the pattern you'll use for every PyTorch project. Memorize it.

In [5]

```

1  # === THE UNIVERSAL PYTORCH TRAINING LOOP ===
2  import torch
3  import torch.nn as nn
4  import torch.optim as optim
5
6  # 1. Define model
7  class Net(nn.Module):
8      def __init__(self, input_dim, hidden_dim, output_dim):
9          super().__init__()
10         self.net = nn.Sequential(
11             nn.Linear(input_dim, hidden_dim),
12             nn.BatchNorm1d(hidden_dim),
13             nn.ReLU(),
14             nn.Dropout(0.2),
15             nn.Linear(hidden_dim, output_dim),
16             nn.Sigmoid()
17         )
18     def forward(self, x):
19         return self.net(x)
20
21 # 2. Create model, optimizer, loss
22 model = Net(2, 32, 1)
23 optimizer = optim.Adam(model.parameters(), lr=0.001)
24 criterion = nn.MSELoss()
25
26 # 3. Training loop (MEMORIZE THIS PATTERN)
27 def train_epoch(model, X_train, y_train, optimizer, criterion):
28     model.train() # Enable dropout/batchnorm training mode
29     predictions = model(X_train) # Forward pass
30     loss = criterion(predictions, y_train) # Compute loss
31     optimizer.zero_grad() # Clear old gradients
32     loss.backward() # Compute new gradients (autograd!)
33     optimizer.step() # Update weights
34     return loss.item()
35
36 # 4. Validation (NO gradients!)
37 def validate(model, X_val, y_val, criterion):
38     model.eval() # Disable dropout/batchnorm
39     with torch.no_grad(): # No gradient computation
40         predictions = model(X_val)
41         loss = criterion(predictions, y_val)
42         accuracy = ((predictions > 0.5).float() == y_val).float().mean()
43     return loss.item(), accuracy.item()
44
45 print("Template ready. Modify input_dim, hidden_dim, output_dim for your task.")

```

## 6. Debugging Checklist — When Your Model Won't Train

In order of likelihood:

- **Learning rate too high/low?** → Try 0.001 (Adam) or 0.01 (SGD) first
- **Data normalized?** →  $X = (X - X.mean()) / X.std()$
- **Correct loss function?** → MSE for regression, BCE for classification
- **Can it overfit 1 batch?** → If not, architecture or code bug
- **Gradients flowing?** → Check `param.grad` is not None or all zeros
- **Labels correct?** → Plot X vs y, sanity check
- **BatchNorm before activation** → Linear → BN → ReLU, never Linear → ReLU → BN

In [6]

```

1 # === QUICK DEBUGGING TOOLS ===
2
3 def check_gradients(model):
4     "Check if gradients are flowing properly."
5     print("Gradient check:")
6     for name, param in model.named_parameters():
7         if param.grad is not None:
8             grad_mean = param.grad.abs().mean().item()
9             grad_max = param.grad.abs().max().item()
10            status = "OK" if grad_mean > 1e-7 else "DEAD"
11            print(f" {name:30s} mean={grad_mean:.2e} max={grad_max:.2e} [{status}]")
12        else:
13            print(f" {name:30s} NO GRADIENT!")
14
15 def check_overfit_one_batch(model, X_batch, y_batch, epochs=200):
16     "If model cannot overfit 1 batch, something is wrong."
17     optimizer = optim.Adam(model.parameters(), lr=0.01)
18     criterion = nn.MSELoss()
19     for i in range(epochs):
20         model.train()
21         loss = criterion(model(X_batch), y_batch)
22         optimizer.zero_grad()
23         loss.backward()
24         optimizer.step()
25     print(f"Final loss after {epochs} epochs on 1 batch: {loss.item():.6f}")
26     if loss.item() > 0.1:
27         print("WARNING: Cannot overfit one batch! Check architecture/code.")
28     else:
29         print("OK: Model can overfit. Training issue is elsewhere.")
30
31 print("Use check_gradients(model) after one backward pass.")
32 print("Use check_overfit_one_batch(model, X[:32], y[:32]) to sanity check.")

```

## 7. VAE — The Generative Model Pattern

```

1 Encoder:  $x \rightarrow \mu, \log(\sigma^2)$ 
2 Reparameterize:  $z = \mu + \sigma \cdot \epsilon, \epsilon \sim N(0,1)$ 
3 Decoder:  $z \rightarrow x_{\text{reconstructed}}$ 
4 Loss: reconstruction_error + KL_divergence

```

In [7]

```

1 # === VAE TEMPLATE (PyTorch) ===
2
3 class VAE(nn.Module):
4     def __init__(self, input_dim, hidden_dim, latent_dim):
5         super().__init__()
6         # Encoder
7         self.encoder = nn.Sequential(
8             nn.Linear(input_dim, hidden_dim), nn.ReLU(),
9             nn.Linear(hidden_dim, hidden_dim // 2), nn.ReLU(),
10        )
11        self.fc_mu = nn.Linear(hidden_dim // 2, latent_dim)
12        self.fc_logvar = nn.Linear(hidden_dim // 2, latent_dim)
13
14        # Decoder
15        self.decoder = nn.Sequential(
16            nn.Linear(latent_dim, hidden_dim // 2), nn.ReLU(),
17            nn.Linear(hidden_dim // 2, hidden_dim), nn.ReLU(),
18            nn.Linear(hidden_dim, input_dim), nn.Sigmoid(),
19        )
20
21        def encode(self, x):
22            h = self.encoder(x)
23            return self.fc_mu(h), self.fc_logvar(h)
24
25        def reparameterize(self, mu, logvar):
26            std = torch.exp(0.5 * logvar)
27            eps = torch.randn_like(std) # Sample from N(0,1)
28            return mu + std * eps # The trick!
29
30        def decode(self, z):
31            return self.decoder(z)
32
33        def forward(self, x):
34            mu, logvar = self.encode(x)
35            z = self.reparameterize(mu, logvar)
36            return self.decode(z), mu, logvar
37
38        def vae_loss(x_hat, x, mu, logvar, beta=1.0):
39            recon = nn.functional.mse_loss(x_hat, x, reduction='sum')
40            kl = -0.5 * torch.sum(1 + logvar - mu.pow(2) - logvar.exp())
41            return recon + beta * kl
42
43        # Generate new samples: z = torch.randn(n, latent_dim) → decoder(z)
44        print("VAE template ready. Adjust dims for your data.")

```

## 8. Deployment — Gradio in 10 Lines

```

1 import gradio as gr
2
3 def predict(input_data):
4     # Your model inference here
5     return model(input_data)
6
7 demo = gr.Interface(
8     fn=predict,
9     inputs=gr.Slider(-3, 3),
10    outputs=gr.Plot()
11 )
12 demo.launch() # Opens a web app!

```

**Deploy to Hugging Face Spaces:** 1. Create repo at [huggingface.co/new-space](https://huggingface.co/new-space) 2. Upload: `app.py`, `model.pth`, `requirements.txt` 3. Done — you have a URL to share

## 9. The 7 Curriculum Truths — Quick Reference

| # | Truth                      | One-Liner                                                       |
|---|----------------------------|-----------------------------------------------------------------|
| 1 | Universal Approximation    | One wide layer CAN fit anything; depth is more EFFICIENT        |
| 2 | ReLU Changed Everything    | Gradient = 0 or 1, never shrinks → deep nets can train          |
| 3 | Overparameterization Helps | More params than data? GD + structure = implicit regularization |
| 4 | NNs Don't Understand       | They minimize loss. They find statistics, not meaning           |
| 5 | Backprop = Chain Rule      | Dynamic programming for gradients. Nothing mystical             |
| 6 | Geometry Matters           | Flat minima generalize. Sharp minima don't                      |
| 7 | Capacity != Performance    | Init + normalization + data quality beat raw size               |

## 10. What to Learn Next

| Topic                   | Why                           | Best Starting Resource                                         |
|-------------------------|-------------------------------|----------------------------------------------------------------|
| <b>CNNs</b>             | Image recognition at scale    | <a href="#">fast.ai Practical Deep Learning, Lessons 1-3</a>   |
| <b>Transformers</b>     | Foundation of modern LLMs     | <a href="#">Andrej Karpathy "Let's build GPT" (YouTube)</a>    |
| <b>Diffusion Models</b> | State-of-art image generation | <a href="#">Lilian Weng blog: "What are Diffusion Models?"</a> |
| <b>Fine-tuning LLMs</b> | Practical GenAI applications  | <a href="#">Hugging Face PEFT documentation</a>                |
| <b>MLOps</b>            | Production deployment         | <a href="#">Made With ML (madewithml.com)</a>                  |

---

*Keep this notebook open while you work. Every pattern you need is here.*

# DEEP LEARNING QUICK REFERENCE CARD

The 7 Truths | Key Formulas | PyTorch Patterns | Debugging Checklist

## THE 7 CURRICULUM TRUTHS

- 1 Universal Approximation**  
1 wide layer CAN fit anything; depth is more efficient
- 2 ReLU Changed Everything**  
Gradient = 0 or 1, never vanishes in chains
- 3 Overparameterization Helps**  
More params than data? GD + structure = regularization
- 4 NNs Minimize Loss**  
Statistics, not understanding or meaning
- 5 Backprop = Chain Rule**  
 $dL/dW = dL/da * da/dz * dz/dW$
- 6 Geometry Matters**  
Flat minima generalize; sharp minima don't
- 7 Capacity != Performance**  
Init + normalization + data quality dominate

## KEY FORMULAS

**Neuron:**  $output = activation(W \cdot x + b)$

**ReLU:**  $max(0, z)$  gradient: 0 or 1

**Sigmoid:**  $1/(1+e^{-z})$  max gradient: 0.25

**MSE Loss:**  $L = (1/N) * \sum((y - \hat{y})^2)$

**Backprop:**  $dL/dW = dL/da * da/dz * x$

**He Init:**  $std = \sqrt{2 / fan\_in}$

**Xavier:**  $std = \sqrt{1 / fan\_in}$

**VAE Loss:**  $recon\_error + beta * KL\_divergence$

**Reparam:**  $z = \mu + \sigma * \epsilon, \epsilon \sim N(0,1)$

## PYTORCH TRAINING LOOP

```
model.train()           # enable dropout/BN
pred = model(X_train)    # forward pass
loss = criterion(pred, y) # compute loss
optimizer.zero_grad()    # clear gradients
loss.backward()          # backprop
optimizer.step()         # update weights

model.eval()             # disable dropout
with torch.no_grad():    # no gradients
    val_pred = model(X_val)
```

## ARCHITECTURE TEMPLATE

```
nn.Sequential(
    nn.Linear(in, hidden),
    nn.BatchNorm1d(hidden), # normalize
    nn.ReLU(),              # activate
    nn.Dropout(0.2),        # regularize
    nn.Linear(hidden, out),
    nn.Sigmoid()            # for binary
)
```

## OPTIMIZERS

**Adam**  $lr=0.001$   
Default. Adapts per-param. Fast convergence.

**SGD**  $lr=0.01$   
Simple. Needs momentum for best results.

**SGD+Mom**  $lr=0.01, m=0.9$   
SGD + inertia. Often best for generalization.

## DEBUGGING CHECKLIST

- 1. Learning rate too high/low?**  
Try 0.001 (Adam) or 0.01 (SGD)
- 2. Data normalized?**  
 $X = (X - \text{mean}) / \text{std}$
- 3. Correct loss function?**  
MSE=regression, BCE=classification
- 4. Can it overfit 1 batch?**  
If not: architecture or code bug
- 5. Gradients flowing?**  
Check `param.grad` is not None/zero
- 6. Labels correct?**  
Plot X vs y, sanity check
- 7. BatchNorm before activation?**  
Linear -> BN -> ReLU (correct)

## INITIALIZATION GUIDE

|         |                    |                     |
|---------|--------------------|---------------------|
| He init | ReLU / LeakyReLU   | $\sqrt{2/fan\_in}$  |
| Xavier  | Sigmoid / Tanh     | $\sqrt{1/fan\_in}$  |
| Zeros   | Biases only        | 0.0                 |
| NEVER   | Zeros for weights! | Network won't learn |

## NOTEBOOK INDEX

|       |                            |
|-------|----------------------------|
| w1_01 | Single neuron from scratch |
| w1_02 | Backpropagation engine     |
| w1_03 | Multi-layer + UAT          |
| w2_01 | PyTorch first network      |
| w2_02 | Init, BN, loss landscapes  |
| w3_01 | Autoencoder + VAE (NumPy)  |
| w3_02 | PyTorch VAE on images      |
| w3_03 | Deploy with Gradio         |